第 1 章 Python 概览
1.1 Python 简介
1.2 Python 的执行机制
1.3 编译型与解释型语言的区别
1.4 Python 的版本与环境
推荐安装方式
1.5 Python 解释器
1.6 Python 程序的运行方法
方一：命令行执行
方二：交互式模式
方三：使用 Jupyter Notebook
1.7 命令行基础操作
第 2 章 基础语法与数据类型
2.1 概述
2.2 标识符与命名规则
命名规则：
查看 Python 关键字
2.3 缩进与语句块
2.4 注释
单行注释
多行注释（或文档字符串）
2.5 变量与赋值
2.6 基本数据类型
2.7 类型转换
2.8 运算符与表达式
算术运算符
比较运算符
逻辑运算符
成员运算符与身份运算符
2.9 输入与输出
输入：`input()`
输出：`print()`
2.10 代码示例：简单计算器
第 3 章 控制结构（Control Structures）—— 决策与循环（Decision & Looping）
3.1 概述（Overview）
3.2 if 单分支语句（Single-Branch if Statement）
基本格式（Syntax）
3.3 if-else 双分支（Two-Way Selection）
3.4 嵌套 if（Nested if Statement）
3.5 双 if 与 if-elif-else 比较
(1) 双 if（Multiple Independent ifs）
(2) if-elif-else（Mutually Exclusive Branches）
3.6 条件表达式（三元运算符, Ternary Operator）
3.7 while 循环（while Loop）
3.8 for 循环（for Loop）
常见可迭代对象（Iterable Types）：
3.9 循环类型比较（Comparison of Loop Types）
3.10 循环控制语句（Loop Control Statements）
3.11 循环中的 else（Loop with else Clause）
3.12 综合案例：质数检测（Prime Number Check）
第 4 章 函数与模块化编程
4.1 概述（Overview）

4.2 函数的定义与调用（Defining and Calling Functions）
基本语法（Syntax）
4.3 return 语句（Return Statement）
功能
示例
多个返回值（Multiple Returns）
- **什么时候需要使用 `return`？（When to Use `return`）**
4.4 实参与形参（Arguments vs Parameters）
定义区分
- **什么时候要加入实参？（When to Pass Arguments）**
4.5 参数类型（Parameter Types）
4.6 综合示例：return 与实参的配合使用
4.7 补充技巧：不返回值 vs 返回值的区别
第 5 章 数据结构与序列类型
5.1 概述（Overview）
5.2 列表（List）
创建列表（Create List）
访问与修改（Access & Modify）
添加与删除元素（Add & Remove Elements）
列表切片（List Slicing）
列表常用方法（List Methods）
5.3 元组（Tuple）
创建与访问
元组 vs 列表
5.4 字符串（String）
基本操作
切片与索引
拼接与格式化（Concatenation & Formatting）
5.5 集合（Set）
创建（Create）
集合运算（Set Operations）
常用方法
5.6 字典（Dictionary）
创建字典（Create Dict）
访问字典
修改字典（Modify）
删除键值（Delete）
遍历字典（Iteration）
常用方法
5.7 序列通用操作（Common Sequence Operations）
5.8 列表推导式（List Comprehension）
5.9 嵌套数据结构（Nested Structures）
5.10 深浅拷贝（Shallow vs Deep Copy）
浅拷贝（Shallow Copy）
深拷贝（Deep Copy）
5.11 小结（Summary）
第 6 章 文件与异常处理
6.1 概述（Overview）
6.2 文件路径与模式（File Paths & Modes）
6.3 文件读取（Reading Files）
方式 1：一次性读取整个文件
方式 2：逐行读取
方式 3：一次读取一行

6.4 文件写入（Writing Files）
写入字符串
追加内容
6.5 使用 with 管理文件（Using `with` Statement）
6.6 文件与路径操作（File and Path Operations）
6.7 异常（Exceptions）
定义
6.8 异常处理（Exception Handling）
基本格式
6.9 else 和 finally 子句（`else` & `finally` Blocks）
6.10 抛出异常（Raising Exceptions）
6.11 自定义异常（Custom Exception）
6.12 将文件操作与异常结合使用（File + Exception）
6.13 综合案例：学生成绩记录系统（Example Case）
第 7 章 面向对象编程
7.1 概述（Overview）
7.2 类与对象（Classes & Objects）
关键概念
定义类（Defining a Class）
创建对象（Creating an Object）
7.3 `__init__` 构造方法（Constructor）
7.4 `self` 关键字（The `self` Keyword）
7.5 封装（Encapsulation）
私有属性（Private Attributes）
7.6 继承（Inheritance）
`super()` 调用父类方法（Calling Parent Methods）
7.7 多态（Polymorphism）
7.8 类属性与实例属性（Class vs Instance Attributes）
7.9 特殊方法（Magic Methods / Dunder Methods）
7.10 静态方法与类方法（Static & Class Methods）
类方法（`@classmethod`）
静态方法（`@staticmethod`）
7.11 封装 + 继承 + 多态 综合案例
7.12 小结（Summary）
第8章 魔法函数与对象行为控制（Magic Functions & Object Behavior Control）
8.1 什么是魔法函数（Magic Functions）
8.2 基本分类与作用
8.3 对象初始化与表示方法
`__init__()` — 初始化对象属性
`__repr__()` 与 `__str__()` — 打印与显示
8.4 算术与比较方法
8.5 容器行为控制
8.6 可调用对象（`__call__`）
8.7 上下文管理方法（`__enter__` 和 `__exit__`）
8.8 迭代行为（`__iter__` 与 `__next__`）
8.9 属性访问控制（`__getattr__` / `__setattr__` / `__delattr__`）
8.10 对象生命周期方法
`__del__()` — 对象销毁时触发。
`__copy__` 与 `__deepcopy__`
8.11 进阶示例：打造一个能“像列表+函数”的对象
8.12 小结
第 9 章 文件与数据库

9.1 概述（Overview）
9.2 文件与数据库的区别（Files vs Databases）
9.3 文件数据格式回顾（File Format Review）
1 文本文件（Text Files）
2 CSV 文件（Comma-Separated Values）
3 JSON 文件（JavaScript Object Notation）
9.4 数据库存储（Databases）
1 什么是数据库（What is a Database）
2 常见数据库类型
9.5 SQLite 简介（Introduction to SQLite）
9.6 SQLite 基本操作（Using `sqlite3` Module）
连接数据库
创建表（Create Table）
插入数据（Insert Data）
查询数据（Select Data）
更新与删除（Update & Delete）
关闭连接（Close Connection）
9.7 使用参数化查询（Parameterized Queries）
9.8 事务（Transactions）
9.9 上下文管理简化操作（Using `with` Statement）
9.10 将数据库结果保存为 JSON 文件
9.11 综合案例：学生管理系统（Student Management System）
9.12 导出为 CSV 文件（Export as CSV）
9.13 使用 Pandas 访问数据库（Using Pandas with Databases）
9.14 小结（Summary）
第 10 章 图形用户界面编程
10.1 概述（Overview）
10.2 Tkinter 简介（About Tkinter）
10.3 创建基本窗口（Creating a Basic Window）
10.4 添加控件（Adding Widgets）
示例：添加标签与按钮
10.5 布局管理（Layout Management）
例：使用 `grid()` 实现登录表单
10.6 文本输入与输出（Text and Input Widgets）
获取文本框输入
设置标签文字
清空输入框
10.7 复选框与单选框（Checkbutton & Radiobutton）
10.8 列表框（Listbox）与滚动条（Scrollbar）
10.9 弹出对话框（Messagebox）
10.10 菜单（Menus）
10.11 画布（Canvas）绘图
10.12 文件选择对话框（File Dialog）
10.13 事件绑定（Event Binding）
10.14 案例：简单计算器（Simple Calculator）
10.15 综合案例：学生信息录入系统（Student Info Form）
10.16 小结（Summary）
第 11 章：函数式与高级特性（Functional and Advanced Features）
11.1 高阶函数（Higher-order Functions）
定义（Definition）
1. map() 函数
2. filter() 函数
3. reduce() 函数

11.2 生成器与惰性计算（Generators and Lazy Evaluation）
定义（Definition）
11.3 迭代器协议（Iterator Protocol）
概念（Concept）
11.4 闭包内部结构解析（Closure Internal Structure）
定义（Definition）
11.5 装饰器应用与封装模式（Decorator Application and Encapsulation Patterns）
概念（Concept）
应用场景（Use Cases）
11.6 内省与动态类型特性（Introspection and Dynamic Typing Features）
定义（Definition）
第 12 章：模块、包与环境（Modules, Packages and Environments）
12.1 模块导入机制（Module Import Mechanism）
定义（Definition）
导入方式（Import Methods）
模块搜索路径（Module Search Path）
12.2 自定义包与目录结构（Custom Packages and Directory Structure）
定义（Definition）
包结构（Package Structure）示例：
`__init__.py` 的作用
12.3 虚拟环境（Virtual Environments）
定义（Definition）
1. venv 虚拟环境
2. conda 环境
3. 环境依赖管理（Dependency Management）
12.4 第三方包管理（Third-party Package Management with pip）
pip 常用命令（Common pip Commands）
离线与镜像源（Offline & Mirror Sources）
12.5 模块缓存与命名空间原理（Module Caching and Namespace Principles）
模块缓存机制（Module Caching）
命名空间原理（Namespace Principle）
第 13 章：调试与测试（Debugging and Testing）
13.1 调试流程与工具（Debug Workflow and Tools）
定义（Definition）
常见错误类型扩展表（Extended Table of Common Python Error Types
按类别层次分类（Categorization by Class Hierarchy）
示例：如何捕获常见异常（Example: Catching Common Exceptions）
IDE 调试（Visual Studio Code / PyCharm）
13.2 打印调试与断点调试（Print Debugging and Breakpoint Debugging）
打印调试（Print Debugging）
断点调试（Breakpoint Debugging）
13.3 assert 语句与防御性编程（Assertions and Defensive Programming）
定义（Definition）
防御性编程（Defensive Programming）
13.4 单元测试基础（Unit Testing Basics）
定义（Definition）
unittest 示例（Example）
常用断言方法（Common Assertions）
pytest 框架
13.5 日志记录与错误追踪（Logging and Error Tracking）
定义（Definition）
基础示例（Basic Example）
日志级别（Logging Levels）

日志输出到文件（Output to File）
错误追踪（Error Tracking with Traceback）
第 14 章：进阶主题（Advanced Topics）
14.1 生成器与迭代器（Generators and Iterators）
定义（Definition）
迭代器（Iterator）
生成器（Generator）
生成器表达式（Generator Expression）
优点（Advantages）
14.2 装饰器（Decorators）
定义（Definition）
基本语法（Basic Syntax）
带参数的装饰器（Decorators with Arguments）
常见用途（Common Use Cases）
14.3 匿名函数与函数式编程（Lambda and Functional Programming）
匿名函数（Lambda Expressions）
函数式工具（Functional Tools）
1. `map()`
2. `filter()`
3. `reduce()`
14.4 迭代工具与生成器模块（Itertools and Generator Utilities）
itertools 模块简介
示例：组合生成
14.5 上下文管理器（Context Managers）
定义（Definition）
示例
自定义上下文管理器
14.6 元类与动态类创建（Metaclasses and Dynamic Class Creation）
定义（Definition）
基本示例
用途总结
14.7 协程与异步编程（Coroutines and Async Programming）
定义（Definition）
示例：异步函数（Async Function）
应用场景（Applications）
14.8 内省与反射（Introspection and Reflection）
定义（Definition）
常用函数
14.9 序列化与反序列化（Serialization and Deserialization）
定义（Definition）
使用 `json` 模块
使用 `pickle` 模块
14.10 正则表达式与文本处理（Regular Expressions and Text Processing）
正则基础（Basics）
常见正则模式（Common Patterns）
第 15 章：递归函数（Recursive Functions）
15.1 递归的基本概念（Basic Concept of Recursion）
定义（Definition）
递归的两个基本条件
递归过程示意
15.2 递归的调用栈（Call Stack）
15.3 常见递归实例（Typical Recursive Examples）
1. 阶乘（Factorial）

2. 斐波那契数列（Fibonacci Sequence）
3. 列表求和（List Sum）
4. 反向打印字符串（Reverse String）
5. 目录遍历（Directory Traversal）
15.4 递归与迭代（Recursion vs. Iteration）
15.5 Python 的递归限制（Recursion Limit）
查看或修改递归深度限制：
15.6 尾递归与优化（Tail Recursion and Optimization）
尾递归（Tail Recursion）
15.7 分治算法与递归（Divide and Conquer Algorithms）
示例：二分查找（Binary Search）
15.8 使用递归构建复杂结构（Recursive Structures）
树的遍历（Tree Traversal）
15.9 总结与实践（Summary and Practice）
知识要点总结
第 16 章：排序算法（Sorting Algorithms）
16.1 排序的基本概念（Basic Concept of Sorting）
定义（Definition）
排序分类（Types of Sorting）
常见排序算法汇总
16.2 冒泡排序（Bubble Sort）
算法思想
示例代码
优化版本（提前终止）
16.3 选择排序（Selection Sort）
算法思想
示例代码
复杂度分析
16.4 插入排序（Insertion Sort）
算法思想
示例代码
特点
16.5 快速排序（Quick Sort）
算法思想
代码实现
特征与优化
16.6 归并排序（Merge Sort）
算法思想
示例实现
特点
16.7 堆排序（Heap Sort）
算法思想
16.8 Python 内置排序（Built-in Sorting）
关键参数
示例：按字符串长度排序
16.9 各算法性能对比总结
第 17 章：查找算法（Searching Algorithms）
17.1 查找的基本概念（Basic Concept of Searching）
定义（Definition）
查找问题的形式化表达
查找算法的分类
17.2 线性查找（Linear Search）
算法思想

代码实现
分析
优化方式：提前判断
17.3 二分查找（Binary Search）
算法思想
递归实现
迭代实现
分析
可视化搜索过程示例
17.4 哈希查找（Hash Search）
算法思想
示例：Python 字典
原理说明
哈希表性能
自定义哈希查找示例
17.5 查找算法性能对比
17.6 二分查找的扩展应用（Binary Search Extensions）
1. 查找插入位置
2. 查找最左或最右匹配（重复元素情况）
3. 在区间中应用二分
17.7 使用 Python 标准库查找
1. `in` 运算符
2. `index()` 方法
3. `bisect` 模块（二分查找工具）
Python 内置查找的特点
17.8 小结与练习
要点回顾
练习题
第 18 章：搜索与排序的综合应用
18.1 引言（Introduction）
18.2 从排序到查找的关系（Relation Between Sorting and Searching）
18.3 实例：学生成绩管理系统（Sorting + Searching）
任务描述
示例代码
说明
18.4 案例：商品库存系统（Product Stock System）
需求场景
结构设计
实现示例
要点
18.5 案例：数据库查询优化思想
简化版数据库索引模拟
18.6 性能比较实验（Performance Analysis）
数据规模实验
结果示例（大致）
18.7 实际开发中的技巧（Practical Tips）
第19章：数据分析与扩展库入门 (Data Analysis and Extension Libraries)
19.1 NumPy 基础（NumPy Basics）
NumPy 简介
NumPy 的核心对象：ndarray
数组的基本属性
向量化操作（Vectorized Operations）
常用函数

19.2 Pandas DataFrame 操作（Pandas DataFrame Operations）
Pandas 概述
创建 DataFrame
基础操作
条件筛选与排序
新增与删除列
19.3 Matplotlib 绘图入门（Matplotlib Visualization Basics）
Matplotlib 简介
示例：绘制折线图
柱形图与饼图
美化样式
19.4 文件与数据结合分析练习（Data Analysis with Files）
读取 CSV 文件
分组与聚合分析
保存数据
综合示例：绘制成绩分布图
第20章：综合实践项目（Comprehensive Practice Projects）
20.1 学生成绩管理系统（Student Score Management System）
项目目标
示例数据（students.csv）
代码实现
20.2 文件批处理与日志分析（File Batch Processing & Log Analysis）
场景
示例日志文件（logs/）
代码实现
进阶分析（错误类型统计）
20.3 简单爬虫与数据提取（Basic Web Crawler & Data Extraction）
项目目标
代码实现
延伸任务
20.4 CLI 实用工具脚本开发（CLI Utility Script Development）
任务描述
代码示例
附录（Appendix）
一、常见错误类型与解决方案（Common Errors & Solutions）
二、基础数据操作（Data Manipulation Basics）
1. 列表（List）
2. 字典（Dictionary）
3. 集合（Set）
4. 元组（Tuple）
5. 字符串（String）
三、数据排序与修改技巧（Sorting and Data Modification）
1. 根据条件排序
2. 多条件排序
3. 列表批量修改
4. Pandas 中的排序与修改
四、内置函数速查表（Built-in Functions Quick Reference）
五、魔法方法一览（Magic Methods Overview）
六、代码规范（Python PEP 8 Style Guide）
命名参考
格式规则
七、推荐学习资源（Recommended Learning Resources）
官方与权威文档

学习平台与练习网站
推荐书籍

第 1 章 Python 概览

1.1 Python 简介

Python 是一种解释型、交互式、高级动态编程语言，由 Guido van Rossum 于 1991 年正式发布。
其设计哲学是“简单明了、可读性强”，强调代码可维护性和开发效率。

主要特点：

- 语法简洁、接近自然语言；
- 拥有强大的标准库；
- 支持多种编程范式：面向对象、过程式、函数式编程；
- 跨平台，可运行于 Windows、macOS、Linux 等操作系统。

常见应用领域：

- 数据分析与科学计算（NumPy、Pandas、SciPy）
- Web 开发（Django、Flask）
- 自动化脚本与测试
- 人工智能与机器学习（TensorFlow、PyTorch）
- 网络爬虫、系统运维、工具开发等。

1.2 Python 的执行机制

Python 是典型的 **解释型语言**。与 C/C++ 等编译型语言不同，它不将源代码直接编译为机器码，而是：

1. 由解释器读取 **.py 源文件**；
2. 编译为 **字节码（.pyc）**；
3. 字节码由 Python 虚拟机（PVM）解释执行。

因此，Python 程序无需手动编译，修改后可直接运行。
运行速度上略慢于编译型语言，但灵活性和开发速度更高。

1.3 编译型与解释型语言的区别

分类	编译型语言（C/C++）	解释型语言（Python）
执行方式	编译为机器码后运行	逐行解释执行
执行速度	较快	略慢

分类	编译型语言（C/C++）	解释型语言（Python）
可移植性	与平台绑定	跨平台
调试与修改	需重新编译	可直接运行

Python 实际是“半编译半解释”的语言：先编译为字节码，再由解释器运行。

1.4 Python 的版本与环境

目前主流版本为 Python 3.x，Python 2 已于 2020 年停止维护。
建议使用 Python 3.9 或以上版本以获得更好的性能与库兼容性。

可通过以下命令检查版本：

```
python --version
# 或
python3 --version
```

推荐安装方式

- 官方网站下载安装包（https://www.python.org）
- 使用 Anaconda 发行版（内置科学计算工具）
- macOS/Linux 系统可直接使用包管理器（如 apt、brew、yum）

1.5 Python 解释器

常见解释器包括：

- **CPython**（官方默认实现）
- **PyPy**（基于 JIT 加速的实现）
- **Jython**（运行在 JVM 上的实现）
- **IronPython**（基于 .NET 的实现）

CPython 是最常用的版本，通常也是我们默认使用的 “python” 命令对应的解释器。

1.6 Python 程序的运行方法

方一：命令行执行

将以下代码保存为 `hello.py`：

```
print("Hello, Python!")
```

然后在终端运行：

```
python hello.py
```

方二：交互式模式

启动解释器后可直接执行语句：

```
python
>>> print("Hello, world!")
Hello, world!
```

方三：使用 Jupyter Notebook

适合教学、实验与交互式脚本，可通过：

```
jupyter notebook
```

在浏览器界面逐行执行代码。

1.7 命令行基础操作

在学习 Python 前，掌握基本终端命令会提高开发效率。常见命令包括：

命令	功能	示例
`pwd`	显示当前工作路径	`pwd`
`ls` / `dir`	查看目录内容	`ls`
`cd`	切换目录	`cd project`
`mkdir`	创建目录	`mkdir src`
`rm` / `del`	删除文件	`rm test.py`
`python file.py`	执行 Python 脚本	`python main.py

第 2 章 基础语法与数据类型

2.1 概述

本章介绍 Python 程序设计的基础语法规则及常用数据类型。掌握这些要点是理解更高级概念的前提。

Python 的语法特点：

- 通过 **缩进** 体现语句块结构；
- 强调 **可读性**；
- 使用 **动态类型系统**，无需声明变量类型；

- 一切数据皆对象。

2.2 标识符与命名规则

在 Python 中，标识符用于命名变量、函数、类、模块等。

命名规则：

1. 只能由 **字母、数字、下划线（_）** 组成；
2. 不能以数字开头；
3. **区分大小写**；
4. 不可与 **关键字** 同名；
5. 建议遵循 [PEP 8 命名规范]：

类型	约定示例
变量	`user_name`
常量	`MAX_SIZE`
函数	`get_value()`
类	`StudentInfo`

查看 Python 关键字

```
import keyword
print(keyword.kwlist)
```

2.3 缩进与语句块

Python 使用 **缩进**（indentation）表示语句块，而非大括号 `{}`。
标准缩进为 4 个空格，混用 Tab 与空格会导致错误。

示例：

```
if True:
    print("缩进正确")
else:
    print("缩进错误可能导致语法错误")
```

> 注意：同一层级的语句缩进必须一致。

2.4 注释

注释用于提高代码可读性。Python 支持两种注释方式：

单行注释

以 `#` 开头：

```
# 这是单行注释
```

多行注释（或文档字符串）

```
"""
多行注释可用于模块、函数说明
"""
```

2.5 变量与赋值

变量在首次赋值时自动声明，类型根据右侧值动态确定：

```
name = "Alice"
age = 20
height = 1.68
```

Python 是 **动态类型语言**，无需指定类型，可以随时改变：

```
a = 10
a = "text"
```

可一次声明多个变量：

```
x, y, z = 1, 2, 3
```

2.6 基本数据类型

Python 的常用基础数据类型如下：

类型	示例	说明
整数（`int`）	10, -5	无小数的数字
浮点数（`float`）	3.14, -0.5	含小数
字符串（`str`）	"hello", 'Python'	文本序列
布尔值（`bool`）	True, False	逻辑值
空值（`NoneType`）	None	表示“无”

可以通过 `type()` 查询类型：

```
print(type(3.14))      # <class 'float'>
print(type(True))      # <class 'bool'>
```

2.7 类型转换

常见类型转换函数：

函数	说明	示例
`int(x)`	转为整数	`int(3.9)` → 3
`float(x)`	转为浮点数	`float(2)` → 2.0
`str(x)`	转为字符串	`str(3.14)` → '3.14'
`bool(x)`	转为布尔值	`bool(0)` → False

示例：

```
a = 10
b = "20"
c = int(b) + a    # 字符串转整数
print(c)
```

2.8 运算符与表达式

Python 支持多种运算符。

算术运算符

运算	符号	示例	结果
加	`+`	3 + 5	8
减	`-`	9 - 2	7
乘	`*`	2 * 4	8
除	`/`	10 / 3	3.3333
整除	`//`	10 // 3	3
取余	`%`	10 % 3	1
幂	`**`	2 ** 3	8

比较运算符

符号	含义	示例
`==`	等于	`3 == 3` → True
`!=`	不等于	`3 != 2`
`>`	大于	`5 > 4`
`<`	小于	`1 < 3`
`>=`	大于等于	`5 >= 5`
`<=`	小于等于	`2 <= 8`

逻辑运算符

符号	含义	示例	结果
`and`	逻辑与	`True and False`	False
`or`	逻辑或	`True or False`	True
`not`	逻辑非	`not True`	False

成员运算符与身份运算符

类型	符号	含义
成员运算符	`in`, `not in`	判断是否包含
身份运算符	`is`, `is not`	判断是否为同一对象

```
a = [1, 2, 3]
print(2 in a)      # True
x = y = [4, 5]
print(x is y)      # True
```

2.9 输入与输出

输入：`input()`

```
name = input("请输入姓名：")
print("你好，", name)
```

> 注意：`input()` 返回的类型始终为字符串。

输出：`print()`

```
print("年龄：", 18)
print(f"{name} 的年龄是 {age}")
```

2.10 代码示例：简单计算器

```
# 简单计算器
a = float(input("输入第一个数："))
b = float(input("输入第二个数："))
print("加法结果:", a + b)
print("乘法结果:", a * b)
```

第 3 章 控制结构（Control Structures）—— 决策与循环（Decision & Looping）

3.1 概述（Overview）

在程序设计中，**控制结构（Control Structure）** 用于控制程序的执行流程。
Python 拥有三种基本控制结构（Basic Control Flow Types）：

1. **顺序结构（Sequential Structure）** — 从上到下依次执行语句；
2. **分支结构（Selection / Branching Structure）** — 根据条件执行不同分支；
3. **循环结构（Looping Structure）** — 重复执行特定语句块。

3.2 if 单分支语句（Single-Branch if Statement）

基本格式（Syntax）

```
if 条件表达式（condition）:
    执行语句块（statement_block）
```

示例：

```
age = 18
if age >= 18:
    print("You are an adult.")  # 输出：已成年
```

> ⚠️ 注意（Note）：Python 中 **冒号 ":"** 表示代码块开始，
> **缩进（Indentation）** 为结构的一部分，通常使用 4 个空格。

3.3 if-else 双分支（Two-Way Selection）

当条件为假（False）时执行另一分支：

```
if 条件 (condition):
    语句块1
else:
    语句块2
```

示例：

```
age = 16
if age >= 18:
    print("Allowed to enter.")
else:
    print("Access denied.")
```

3.4 嵌套 if（Nested if Statement）

可在判断内部嵌套二次判断：

```
score = 85
if score >= 60:
    if score >= 90:
        print("Excellent")
    else:
        print("Pass")
else:
    print("Fail")
```

避免过多嵌套，重构为 `if-elif-else` 更清晰。

3.5 双 if 与 if-elif-else 比较

(1) 双 if（Multiple Independent ifs）

每个条件独立检查，不互斥（not mutually exclusive）：

```
score = 95
if score >= 60:
    print("Pass")
if score >= 90:
    print("Excellent")
```

输出：

```
Pass
Excellent
```

(2) if-elif-else（Mutually Exclusive Branches）

仅执行第一个满足条件的分支：

```
score = 95
if score >= 90:
    print("Excellent")
elif score >= 60:
    print("Pass")
else:
    print("Fail")
```

输出：

```
Excellent
```

比较项 (Aspect)	双 if (Two ifs)	if-elif-else
判断次数 (Checks)	每个 if 单独判断	只执行第一个匹配条件
是否独立 (Independence)	互不影响	互斥结构
典型用途 (Use Case)	条件可能同时成立	等级判断场景（Exclusive cases）

3.6 条件表达式（三元运算符, Ternary Operator）

简化条件赋值的单行写法：

```
result = "Adult" if age >= 18 else "Minor"
```

等价于：

```
if age >= 18:
    result = "Adult"
else:
    result = "Minor"
```

3.7 while 循环（while Loop）

基本格式（Syntax）：

```
while 条件（condition）:
    执行语句块（statement_block）
```

执行流程：当条件为真（True）时执行语句块，否则退出循环。

示例：

```
count = 1
while count <= 3:
    print("Count:", count)
    count += 1
```

3.8 for 循环（for Loop）

用于遍历可迭代对象（Iterable Object）。

语法：

```
for 变量（variable）in 可迭代对象（iterable）:
    执行语句块
```

示例：

```
for i in range(5):
    print("Index:", i)
```

常见可迭代对象（Iterable Types）：

- 字符串（String）
- 列表（List）
- 元组（Tuple）
- 集合（Set）
- 字典（Dictionary）
- 序列（Sequence）或 `range()`

3.9 循环类型比较（Comparison of Loop Types）

项目 (Aspect)	for 循环 (for loop)	while 循环 (while loop)
循环对象	用于遍历序列 (Iterate over iterable)	基于条件判断 (Based on condition)
次数控制	固定次数	不确定次数
是否需手动更新变量	否	是
易错点	索引超界	忘记更新变量导致死循环

示例：

```python
# for loop
for i in range(3):
    print("for:", i)

# while loop
i = 0
while i < 3:
    print("while:", i)
    i += 1
```

3.10 循环控制语句（Loop Control Statements）

Python 提供三种循环控制关键字：

关键字 (Keyword)	功能 (Function)	说明
`break`	立即终止循环	Exit the loop immediately
`continue`	跳过当前迭代，继续下一次	Skip the current iteration
`pass`	占位语句，不执行任何操作	Placeholder or empty block

示例：

```python
for i in range(5):
    if i == 2:
        continue      # Skip value 2
    if i == 4:
        break         # Stop the loop
    print("i =", i)
```

输出：

```
i = 0
i = 1
i = 3
```

3.11 循环中的 else（Loop with else Clause）

当循环 **未被 break 中断** 时，`else` 块将被执行。

```python
for i in range(3):
    print(i)
else:
    print("Loop finished normally.")
```

3.12 综合案例：质数检测（Prime Number Check）

```
num = int(input("Enter an integer: "))

if num > 1:
    for n in range(2, num):
        if num % n == 0:
            print(num, "is not a prime number.")
            break
    else:
        print(num, "is a prime number.")
else:
    print("Number should be greater than 1.")
```

输出示例：

```
Enter an integer: 7
7 is a prime number.
```

第 4 章 函数与模块化编程

(Chapter 4: Functions & Modular Programming)

4.1 概述（Overview）

在程序开发中，**函数（Function）** 用于将重复或逻辑性强的代码封装到一个可复用的单元中。
模块化编程（Modular Programming）鼓励把复杂问题拆分成多个小的功能模块。

Python 通过函数（`def`）和模块（`import`）支持这种结构化思想。

4.2 函数的定义与调用（Defining and Calling Functions）

基本语法（Syntax）

```
def 函数名(function_name)(参数列表 parameters):
    语句块 (function_body)
    [return 返回值 (optional return_value)]
```

示例：

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")   # 调用时传入实参 (argument)
```

4.3 return 语句（Return Statement）

功能

`return` 用于 **将函数的计算结果返回给调用方（Return a value to caller）**。

如果函数只是输出结果而不需要返回，则可以省略 `return`。
未写 `return` 时，Python 默认返回 `None`。

示例

```
def add(a, b):
    return a + b  # 返回计算值给调用方

result = add(2, 3)
print(result)  # 输出：5
```

多个返回值（Multiple Returns）

```
def get_user_info():
    return "Alice", 25, "Student"

name, age, role = get_user_info()
```

🔸 什么时候需要使用 `return`？（When to Use `return`）

场景 (Scenario)	是否需要 `return`	示例 (Example)	说明 (Explanation)
只是打印或显示结果	否 (No)	`print()`	函数只执行显示或输出任务
需要在函数外部使用计算结果	✅ 是 (Yes)	`return value`	函数的结果将被其他代码使用
要控制函数提前结束	✅ 是 (Yes)	`return`	用于中断函数，例如错误处理
需要返回多个值	✅ 是 (Yes)	`return x, y, z`	Python 支持多值返回（tuple）

💡 示例：

```
def calculate_discount(price):
    if price < 0:
        return "Invalid price"  # 提前结束函数
    return price * 0.9  # 返回计算值

discounted_price = calculate_discount(100)
```

4.4 实参与形参（Arguments vs Parameters）

定义区分

名称	英文术语	定义
形参	Parameter	定义函数时声明的变量
实参	Argument	调用函数时传入的具体值

示例：

```
def greet(name):        # “name” 是形参 (parameter)
    print("Hello", name)

greet("Bob")            # "Bob" 是实参 (argument)
```

🔸 什么时候要加入实参？ (When to Pass Arguments)

场景 (Scenario)	是否要传实参	示例 (Example)	说明 (Explanation)
函数定义了形参	✅ 是 (Yes)	`add(2, 3)`	调用函数必须传入对应的值
函数需要外部输入数据	✅ 是 (Yes)	`login(username, password)`	实参提供外部输入
函数有默认参数值	可选 (Optional)	`greet()` 或 `greet("Tom")`	未传参使用默认值
函数不用参数	否 (No)	`show_time()`	内部逻辑不依赖外部输入

💡 示例：

```
def greet(name="Guest"):
    print(f"Hello, {name}")

greet()          # 使用默认值 Guest
greet("Alice")   # 传入实参，覆盖默认值
```

4.5 参数类型（Parameter Types）

类型	英文名	示例	说明
位置参数	Positional	`add(2, 3)`	按顺序传入
关键字参数	Keyword	`add(a=2, b=3)`	按名称传入
默认参数	Default	`def greet(name="Guest"):`	有默认值
可变参数	Variable-Length (`*args`, `**kwargs`)	`def display(*args):`	接收多个值

4.6 综合示例：return 与实参的配合使用

```
def calc_area(length, width):
    """计算矩形面积 (Calculate rectangle area)"""
    return length * width    # 返回结果

def show_area(l, w):
    """显示面积结果 (Display area)"""
    area = calc_area(l, w)   # 调用并传入实参
    print(f"Area = {area}")

show_area(5, 10)
```

输出：

```
Area = 50
```

说明：

- `calc_area` 使用 `return` 把结果"交还"给调用者；
- 调用时需要传入实参 `l` 和 `w`；
- 另一函数 `show_area` 使用这个返回值进行显示。

4.7 补充技巧：不返回值 vs 返回值的区别

类型	示例	执行效果
无返回值函数	`def log_info(msg): print(msg)`	直接执行任务，无结果输出到外部
有返回值函数	`def add(x, y): return x + y`	计算后返回值，可复用结果

示例对比：

```
# 无 return：只打印，不返回结果
def print_sum(a, b):
    print(a + b)

# 有 return：返回结果供后续使用
def calc_sum(a, b):
    return a + b

print_sum(3, 4)             # 输出: 7
result = calc_sum(3, 4)     # 保存结果
print("Result =", result)   # 输出: Result = 7
```

第 5 章 数据结构与序列类型

(Chapter 5: Data Structures & Sequence Types)

5.1 概述（Overview）

数据结构（Data Structures） 是程序设计中组织与存储数据的方式。
Python 提供了丰富的内置容器类型（Built-in Container Types），用于 **存储、访问与操作数据集合（data collections）**。

主要序列与集合类型：

类型 (Type)	英文名 (English)	可变性 (Mutable)	是否有序 (Ordered)	示例 (Example)
列表	`list`	✅ 是	✅ 是	`[1, 2, 3]`
元组	`tuple`	❌ 否	✅ 是	`(1, 2, 3)`
字符串	`str`	❌ 否	✅ 是	`"hello"`
集合	`set`	✅ 是	❌ 否	`{1, 2, 3}`
字典	`dict`	✅ 是	✅ (3.7+)	`{'a': 1, 'b': 2}`

5.2 列表（List）

列表是最常用的可变序列（Mutable Sequence），可存储任意类型元素。

创建列表（Create List）

```
fruits = ["apple", "banana", "cherry"]
```

访问与修改（Access & Modify）

```
print(fruits[0])      # 访问第一个元素 -> apple
fruits[1] = "pear"    # 修改第二个元素
```

添加与删除元素（Add & Remove Elements）

```
fruits.append("orange")      # 尾部添加元素
fruits.insert(1, "mango")    # 指定位置插入
fruits.remove("pear")        # 删除指定值
deleted = fruits.pop()       # 删除并返回最后一个元素
```

列表切片（List Slicing）

```
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4])   # [1, 2, 3]
print(nums[:3])    # [0, 1, 2]
print(nums[::2])   # [0, 2, 4]
```

列表常用方法（List Methods）

方法	说明
`.append(x)`	末尾添加元素
`.extend(seq)`	合并序列
`.insert(i, x)`	插入元素
`.remove(x)`	移除指定值
`.pop([i])`	弹出元素
`.sort()` / `sorted()`	排序
`.reverse()`	反转顺序
`.count(x)`	统计出现次数
`.index(x)`	查找索引位置

示例：

```
nums = [3, 1, 4, 2]
nums.sort()
print(nums)   # [1, 2, 3, 4]
```

5.3 元组（Tuple）

元组（Tuple） 是不可变序列（Immutable Sequence），通常用于固定数据集合。

创建与访问

```
colors = ("red", "green", "blue")
print(colors[0])
```

单元素元组要加逗号：

```
single = (5,)
```

元组 vs 列表

特性	元组（Tuple）	列表（List）
可变性	❌ 否	✅ 是
访问速度	快	略慢
内存占用	较小	略大
使用场景	固定数据结构	动态数据集合

5.4 字符串（String）

字符串是不可变序列（Immutable Sequence），由字符（characters）组成。

基本操作

```
s = "Hello World"
print(len(s))       # 11
print(s.upper())    # HELLO WORLD
print(s.lower())    # hello world
```

切片与索引

```
print(s[0])      # 'H'
print(s[1:5])    # 'ello'
```

拼接与格式化（Concatenation & Formatting）

```
name = "Alice"
print("Hello, " + name)
print(f"Hello, {name}")     # f-string 格式化
```

5.5 集合（Set）

集合（Set）是一种无序且不重复的容器（Unordered, Unique Container）。

创建（Create）

```
nums = {1, 2, 3, 3, 2}
print(nums)   # {1, 2, 3}
```

集合运算（Set Operations）

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)   # 并集 union → {1, 2, 3, 4, 5}
print(A & B)   # 交集 intersection → {3}
print(A - B)   # 差集 difference → {1, 2}
print(A ^ B)   # 对称差 symmetric difference → {1, 2, 4, 5}
```

常用方法

方法	功能
`.add(x)`	添加元素
`.remove(x)`	删除指定元素
`.discard(x)`	删除（不存在也不报错）
`.clear()`	清空集合
`.update([...])`	批量添加

5.6 字典（Dictionary）

字典是键值对（Key-Value Pairs）组成的映射结构（Mapping Type）。

创建字典（Create Dict）

```
person = {"name": "Alice", "age": 25, "city": "Beijing"}
```

访问字典

```
print(person["name"])
print(person.get("age"))
```

修改字典（Modify）

```
person["age"] = 26     # 修改
person["email"] = "alice@example.com"  # 新增键值
```

删除键值（Delete）

```
del person["city"]
person.pop("email")
```

遍历字典（Iteration）

```
for key, value in person.items():
    print(key, ":", value)
```

常用方法

方法	功能
`.keys()`	获取所有键
`.values()`	获取所有值
`.items()`	获取键值对
`.get(key, default)`	安全获取值
`.pop(key)`	删除元素
`.update()`	合并字典

5.7 序列通用操作（Common Sequence Operations）

Python 中大多数序列类型（list、tuple、str）都支持以下操作：

操作符	功能	示例
`+`	拼接	`[1,2] + [3,4]` → `[1,2,3,4]`
`*`	重复	`[1,2]*2` → `[1,2,1,2]`
`in`	成员检测	`'a' in 'abc'` → `True`
`len()`	长度	`len([1,2,3])` → `3`
`max()` / `min()`	最大/最小值	`max([1,5,3])` → `5`
`sum()`	求和	`sum([1,2,3])` → `6`

5.8 列表推导式（List Comprehension）

是一种简洁生成列表的方式（Compact Syntax for List Creation）。

```
nums = [1, 2, 3, 4, 5]
squares = [x**2 for x in nums if x % 2 == 1]
print(squares)   # [1, 9, 25]
```

结构：

```
[ expression for variable in iterable if condition ]
```

5.9 嵌套数据结构（Nested Structures）

Python 支持嵌套组合（Nested Combination）：

```
students = [
    {"name": "Alice", "score": [85, 90, 92]},
    {"name": "Bob", "score": [78, 88, 80]},
]
print(students[0]["score"][1])  # 输出 90
```

5.10 深浅拷贝（Shallow vs Deep Copy）

浅拷贝（Shallow Copy）

仅复制最外层引用：

```
import copy
a = [1, [2, 3]]
b = copy.copy(a)
b[1][0] = 99
print(a)   # [1, [99, 3]]
```

深拷贝（Deep Copy）

复制所有层级的新对象：

```
c = copy.deepcopy(a)
c[1][0] = 100
print(a)   # 不受影响
```

5.11 小结（Summary）

数据类型	英文名	可变性	是否有序	典型用途
列表	List	可变	有序	动态数据集合
元组	Tuple	不可变	有序	固定结构

数据类型	英文名	可变性	是否有序	典型用途
字符串	String	不可变	有序	文本处理
集合	Set	可变	无序	唯一元素存储
字典	Dictionary	可变	有序(3.7+)	键值映射关系

第 6 章 文件与异常处理

(Chapter 6: Files & Exception Handling)

6.1 概述（Overview）

在程序中，**文件操作（File Operations）** 与 **异常处理（Exception Handling）** 是两个非常实用的概念。

- 文件操作：用于从文件中读取数据、写入内容或创建记录。
- 异常处理：确保程序在出现错误（Error）时能够优雅处理，而不是直接崩溃。

本章内容帮助你掌握：

1. 如何打开、读取、写入文件；
2. 如何处理异常与错误；
3. 使用 `with` 语句管理资源。

6.2 文件路径与模式（File Paths & Modes）

在 Python 中，通过函数 `open()` 操作文件：

```
open(file, mode, encoding=None)
```

模式 (Mode)	含义 (Meaning)	示例 (Example)
`'r'`	读取（Read）	`open('data.txt', 'r')`
`'w'`	写入（Write），覆盖原内容	`open('new.txt', 'w')`
`'a'`	追加（Append）	`open('log.txt', 'a')`
`'r+'`	读写（Read and Write）	`open('data.txt', 'r+')`
`'b'`	二进制模式（Binary）	`open('img.png', 'rb')`

💡 **提示（Tip）**：
建议总是指定 `encoding='utf-8'`，以避免中文字符出现乱码。

```
f = open("demo.txt", "r", encoding="utf-8")
```

6.3 文件读取（Reading Files）

方式 1：一次性读取整个文件

```
f = open("hello.txt", "r", encoding="utf-8")
content = f.read()
print(content)
f.close()
```

方式 2：逐行读取

```
f = open("hello.txt", "r", encoding="utf-8")
for line in f:
    print(line.strip())
f.close()
```

方式 3：一次读取一行

```
f = open("hello.txt", "r", encoding="utf-8")
line = f.readline()
print(line)
f.close()
```

6.4 文件写入（Writing Files）

写入字符串

```
f = open("output.txt", "w", encoding="utf-8")
f.write("Hello, Python!\n")
f.write("This is a new line.")
f.close()
```

追加内容

```
f = open("output.txt", "a", encoding="utf-8")
f.write("\n追加的文字 (Appended text)")
f.close()
```

6.5 使用 with 管理文件（Using `with` Statement）

`with` 可以自动关闭文件，无需手动调用 `close()`，推荐使用。

```
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)
```

优点：

- 自动释放文件资源；
- 更简洁、安全。

6.6 文件与路径操作（File and Path Operations）

Python 的 `os` 与 `pathlib` 模块可辅助管理文件与文件夹。

```
import os
print(os.getcwd())          # 获取当前目录
os.mkdir("new_folder")      # 创建文件夹
os.remove("data.txt")       # 删除文件
os.listdir(".")             # 列出当前目录下所有文件
```

使用现代的 `pathlib`：

```
from pathlib import Path

p = Path("data.txt")
print(p.exists())           # 判断文件是否存在
print(p.name)               # 文件名
print(p.parent)             # 上一级目录
```

6.7 异常（Exceptions）

定义

异常是程序运行期间发生的错误事件（Error event），会打断程序的正常执行流程。

常见异常类型：

异常类型	原因
`FileNotFoundError`	文件未找到
`ValueError`	数据类型错误
`ZeroDivisionError`	除以零
`TypeError`	类型错误
`IndexError`	列表索引超出范围
`KeyError`	字典键不存在

异常类型	原因
`IOError`	输入输出错误
`ImportError`	模块导入失败

6.8 异常处理（Exception Handling）

Python 使用 `try` - `except` 结构处理异常。

基本格式

```
try:
    可能出错的代码（code that may fail）
except ExceptionType:
    如果出错则执行的代码
```

示例：

```
try:
    x = int(input("请输入数字："))
    print(10 / x)
except ZeroDivisionError:
    print("错误：除数不能为零！")
except ValueError:
    print("错误：输入的不是数字！")
```

6.9 else 和 finally 子句（`else` & `finally` Blocks）

`try` 块可以搭配 `else` 与 `finally` 使用。

```
try:
    f = open("data.txt", "r")
    data = f.read()
except FileNotFoundError:
    print("文件不存在！")
else:
    print("文件读取成功")
finally:
    f.close()
    print("文件已关闭")
```

执行流程：

- `try`：尝试执行；
- `except`：出错时执行；
- `else`：无错误时执行；

- `finally`：无论是否出错都执行（常用于清理资源）。

6.10 抛出异常（Raising Exceptions）

使用 `raise` 主动抛出异常（Manually Raise an Exception）。

```
def divide(a, b):
    if b == 0:
        raise ValueError("除数不能为 0")
    return a / b
```

调用：

```
try:
    result = divide(10, 0)
except ValueError as e:
    print("捕获异常:", e)
```

6.11 自定义异常（Custom Exception）

用户可以通过继承 `Exception` 类自定义异常类型。

```
class NegativeValueError(Exception):
    """自定义异常：输入负数"""
    pass

def check_age(age):
    if age < 0:
        raise NegativeValueError("年龄不能为负数")

try:
    check_age(-5)
except NegativeValueError as e:
    print("捕获自定义异常:", e)
```

6.12 将文件操作与异常结合使用（File + Exception）

```
def read_file(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return f.read()
    except FileNotFoundError:
        print("错误：文件不存在！")
        return None
    except Exception as e:
        print("发生未知错误：", e)
        return None

content = read_file("test.txt")
```

6.13 综合案例：学生成绩记录系统（Example Case）

示例功能：

- 从文件读取学生成绩；
- 添加新数据并写回；
- 使用异常保证稳定性。

```
def update_scores(filename):
    try:
        with open(filename, "r", encoding="utf-8") as f:
            scores = [int(line.strip()) for line in f.readlines()]
    except FileNotFoundError:
        print("文件未找到，创建新文件。")
        scores = []

    try:
        new_score = int(input("请输入新的成绩："))
        scores.append(new_score)
        with open(filename, "w", encoding="utf-8") as f:
            for s in scores:
                f.write(str(s) + "\n")
        print("成绩已保存！")
    except ValueError:
        print("输入无效，请输入数字。")

update_scores("scores.txt")
```

第 7 章 面向对象编程

(Chapter 7: Object-Oriented Programming - OOP)

7.1 概述（Overview）

面向对象编程（Object-Oriented Programming, OOP） 是一种通过对象（objects）来组织代码的编程思想。
相对于过程式编程（Procedural Programming），OOP 更强调：

- **数据（Data）** 与 **行为（Behavior）** 一体化；
- **封装（Encapsulation）**、**继承（Inheritance）**、**多态（Polymorphism）** 三大特性。

核心理念：

> “万物皆对象（Everything is an object）。”

7.2 类与对象（Classes & Objects）

关键概念

名称	英文	含义
类	Class	创建对象的蓝图或模板
对象	Object	类的实例（Instance of a class）
属性	Attribute	对象的数据（变量）
方法	Method	对象的行为（函数）

定义类（Defining a Class）

```
class Person:
    def __init__(self, name, age):
        self.name = name        # 属性
        self.age = age          # 属性

    def greet(self):            # 方法
        print(f"Hello, my name is {self.name}. I am {self.age} years old.")
```

创建对象（Creating an Object）

```
p1 = Person("Alice", 25)
p1.greet()
```

输出：

```
Hello, my name is Alice. I am 25 years old.
```

7.3 `__init__` 构造方法（Constructor）

`__init__` 是一个 **构造函数（constructor）**，用于初始化对象的属性。
它在创建对象时 **自动调用（called automatically）**。

```
class Dog:
    def __init__(self, name):
        self.name = name
        print(f"Dog {self.name} has been created!")
```

调用：

```
d = Dog("Buddy")
```

输出：

```
Dog Buddy has been created!
```

7.4 `self` 关键字（The `self` Keyword）

- `self` 代表 **当前对象自身（the current instance itself）**。
- 类中任何方法的第一个参数必须是 `self`（Pythond 自动传入）。

示例：

```
class Cat:
    def set_name(self, name):
        self.name = name

    def get_name(self):
        return self.name
```

调用：

```
c = Cat()
c.set_name("Kitty")
print(c.get_name())
```

7.5 封装（Encapsulation）

封装（Encapsulation） 指把数据与操作数据的方法绑定在一起，防止外部直接访问内部细节。

私有属性（Private Attributes）

- 使用前缀 `__` 定义私有属性；
- 外部不能直接访问，只能通过方法获取。

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance  # 私有属性

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

使用：

```
account = BankAccount("Alice", 1000)
account.deposit(500)
print(account.get_balance())
# print(account.__balance)  # 错误：无法直接访问
```

7.6 继承（Inheritance）

继承 允许子类（Subclass）**复用父类（Parent Class）**的属性与方法，并可以进行扩展或重写。

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):           # 重写父类方法
        print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")
```

调用：

```
d = Dog()
c = Cat()
d.speak()   # Woof!
c.speak()   # Meow!
```

`super()` 调用父类方法（Calling Parent Methods）

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, grade):
        super().__init__(name)    # 调用父类构造函数
        self.grade = grade
```

7.7 多态（Polymorphism）

多态意为“多种形态”。
同一个接口（方法名），在不同对象中有不同实现。

```
class Bird:
    def sound(self):
        print("Chirp")

class Dog:
    def sound(self):
        print("Bark")

for animal in [Bird(), Dog()]:
    animal.sound()
```

输出：

```
Chirp
Bark
```

7.8 类属性与实例属性（Class vs Instance Attributes）

类型	定义位置	所属对象	共用性
实例属性	构造函数中定义	属于对象实例	各对象独立
类属性	类体中定义	属于整个类	所有实例共享

示例：

```python
class Student:
    school = "Python University"   # 类属性

    def __init__(self, name):
        self.name = name           # 实例属性

s1 = Student("Alice")
s2 = Student("Bob")

print(s1.school, s2.school)   # 都能访问类属性
s1.school = "Another School"  # 为 s1 创建了同名实例属性
print(s1.school, s2.school)
```

7.9 特殊方法（Magic Methods / Dunder Methods）

特殊方法以 `__` 开头结尾，例如：

方法名	功能	示例
`__init__`	构造函数	初始化对象
`__str__`	字符串化	控制 `print(obj)` 输出
`__len__`	返回长度	支持 `len(obj)`
`__add__`	加法运算符重载	定义 `obj1 + obj2` 行为
`__eq__`	等于比较	定义 `obj1 == obj2` 行为

示例：

```python
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)   # 输出 (4, 6)
```

7.10 静态方法与类方法（Static & Class Methods）

类方法（`@classmethod`）

- 第一个参数是 `cls`（代表类本身）；
- 可用于操作类属性。

```
class Student:
    count = 0

    def __init__(self, name):
        self.name = name
        Student.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

print(Student.get_count())  # 0
a = Student("Alice")
b = Student("Bob")
print(Student.get_count())  # 2
```

静态方法（`@staticmethod`）

- 与类或实例无关；
- 用于执行与类逻辑相关、但不依赖实例数据的函数。

```
class MathTool:
    @staticmethod
    def add(a, b):
        return a + b

print(MathTool.add(2, 3))  # 5
```

7.11 封装 + 继承 + 多态 综合案例

```
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h
```

```python
class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14 * self.r * self.r

shapes = [Rectangle(3, 4), Circle(5)]

for s in shapes:
    print("Area:", s.area())
```

输出：

```
Area: 12
Area: 78.5
```

7.12 小结（Summary）

概念	英文	含义
类	Class	对象的模板
对象	Object	类的实例
属性	Attribute	数据成员
方法	Method	行为函数
封装	Encapsulation	隐藏内部数据
继承	Inheritance	复用父类特性
多态	Polymorphism	同接口多实现
`super()`	调用父类方法	
`self`	当前对象引用	
类方法	`@classmethod`	操作类属性方法
静态方法	`@staticmethod`	与类逻辑相关但不依赖实例

第8章 魔法函数与对象行为控制（Magic Functions & Object Behavior Control）

8.1 什么是魔法函数（Magic Functions）

魔法函数（Magic Methods）又被称为**特殊方法（Special Methods）**，它们是 Python 类中以双下划线 `__name__` 命名的特殊函数。

这些方法用于定义对象的 **行为、运算方式、属性访问、迭代能力、上下文环境等**，
使类可以像内置类型一样自然地被操作。

例如：

- 使用 `__add__` 可以使对象支持 `+` 运算；
- 使用 `__len__` 可以让对象支持 `len()`；
- 使用 `__getitem__` 可以让对象支持索引操作。

8.2 基本分类与作用

魔法方法大致分为以下几类：

分类	示例方法	功能说明
对象初始化与表示	`__init__`, `__repr__`, `__str__`, `__del__`	定义对象的创建、打印、销毁等行为
算术与比较操作	`__add__`, `__sub__`, `__mul__`, `__eq__`, `__lt__`	重载运算符，实现对象间计算与比较
容器行为	`__len__`, `__getitem__`, `__setitem__`, `__contains__`	让类像列表或字典一样可索引
可调用对象	`__call__`	让对象能像函数一样被调用
上下文管理	`__enter__`, `__exit__`	支持 `with` 语句资源管理
迭代与生成	`__iter__`, `__next__`	支持 `for` 循环或迭代行为
属性控制	`__getattr__`, `__setattr__`, `__delattr__`	自定义对象属性访问规则
对象表示与复制	`__copy__`, `__deepcopy__`, `__hash__`	复制与哈希行为定义

8.3 对象初始化与表示方法

`__init__()` — 初始化对象属性

定义对象创建时的行为。

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

stu = Student("Alice", 90)
print(stu.name)   # Alice
```

`__repr__()` 与 `__str__()` — 打印与显示

两者的区别在于：

- `__repr__` 用于开发调试输出（正式、准确）
- `__str__` 用于用户友好显示（简洁、美观）

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __repr__(self):
        return f"Student(name={self.name!r}, score={self.score!r})"

    def __str__(self):
        return f"{self.name} 的成绩：{self.score}"

stu = Student("Bob", 87)
print(repr(stu))  # 调试输出
print(str(stu))   # 友好输出
```

8.4 算术与比较方法

使类支持运算符重载。

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"
```

```
v1 = Vector(2, 3)
v2 = Vector(1, 4)
print(v1 + v2)     # Vector(3, 7)
print(v1 - v2)     # Vector(1, -1)
print(v1 == v2)    # False
```

8.5 容器行为控制

让类具备“类列表”或“类字典”的行为。

```
class DataCollection:
    def __init__(self):
        self.data = [10, 20, 30]

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index):
        return self.data[index]

    def __setitem__(self, index, value):
        self.data[index] = value

    def __contains__(self, item):
        return item in self.data

c = DataCollection()
print(len(c))        # 3
print(c[1])          # 20
c[0] = 99
print(99 in c)       # True
```

8.6 可调用对象（`__call__`）

使类实例能像函数一样执行。

```
class Greeter:
    def __init__(self, prefix="Hello"):
        self.prefix = prefix

    def __call__(self, name):
        return f"{self.prefix}, {name}!"

g = Greeter()
print(g("Alice"))   # Hello, Alice!
```

8.7 上下文管理方法（`__enter__` 和 `__exit__`）

允许类使用 `with` 语句进行安全的资源管理。

```
class FileHandler:
    def __init__(self, filename):
        self.filename = filename

    def __enter__(self):
        self.file = open(self.filename, "r", encoding="utf-8")
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

# 使用示例
with FileHandler("data.txt") as f:
    content = f.read()
```

8.8 迭代行为（`__iter__` 与 `__next__`）

自定义可迭代对象，使类能用于 `for` 循环。

```
class Countdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.start <= 0:
            raise StopIteration
        self.start -= 1
        return self.start

for num in Countdown(5):
    print(num)
```

8.9 属性访问控制（`__getattr__` / `__setattr__` / `__delattr__`）

允许对象拦截或修改属性访问行为。

```
class Person:
    def __init__(self, name):
        self.__dict__["name"] = name
```

```
    def __getattr__(self, attr):
        return f"属性 {attr} 不存在"

    def __setattr__(self, attr, value):
        print(f"设置属性: {attr} = {value}")
        self.__dict__[attr] = value

p = Person("Tom")
p.age = 30
print(p.city)     # 属性 city 不存在
```

8.10 对象生命周期方法

`__del__()` — 对象销毁时触发。

```
class Temp:
    def __del__(self):
        print("对象已销毁")

t = Temp()
del t
```

`__copy__` 与 `__deepcopy__`

用于自定义对象复制逻辑（配合 `copy` 模块）。

8.11 进阶示例：打造一个能“像列表+函数”的对象

```
class SmartList:
    def __init__(self, data):
        self.data = list(data)

    def __getitem__(self, idx):
        return self.data[idx]

    def __setitem__(self, idx, val):
        self.data[idx] = val

    def __call__(self, func):
        return [func(x) for x in self.data]

    def __repr__(self):
        return f"SmartList({self.data})"

nums = SmartList([1, 2, 3])
print(nums[1])          # 2
nums[0] = 10
print(nums(lambda x: x * 2))  # [20, 4, 6]
```

此类：

- 像列表一样可索引；
- 像函数一样可被“调用”并批量变换数据；
- 通过魔法函数，使行为高度灵活。

8.12 小结

魔法函数体现了 Python 的**面向对象灵活性**，
它们让自定义类几乎能完全模仿内置类型的行为。

功能领域	关键方法
初始化与表示	`__init__`, `__str__`, `__repr__`
运算与比较	`__add__`, `__sub__`, `__eq__`, `__lt__`, …
容器访问	`__getitem__`, `__setitem__`, `__contains__`
可调用与迭代	`__call__`, `__iter__`, `__next__`
属性与上下文	`__getattr__`, `__setattr__`, `__enter__`, `__exit__`

第 9 章 文件与数据库

(Chapter 9: Files & Databases)

9.1 概述（Overview）

在现代应用中，我们需要 **持久化（persistence）** 数据：
即程序结束后，数据仍能被保存并再次使用。

Python 提供了两种主要的数据持久化方式：

1. **文件（File）** – 轻量级存储，例如文本文件、CSV、JSON。
2. **数据库（Database）** – 结构化数据存储，适用于复杂应用，例如 SQLite、MySQL。

9.2 文件与数据库的区别（Files vs Databases）

对比项	文件（File）	数据库（Database）
数据结构	顺序或文本存储	表格化、关系结构
适用场景	小型或简单项目	大型、结构化项目
查询能力	需自行实现搜索	使用 SQL 高效查询

对比项	文件（File）	数据库（Database）
速度与效率	依赖文件 I/O	通常更高效
示例	`.txt`, `.csv`, `.json`	`.db`, `.sqlite`, MySQL, PostgreSQL

9.3 文件数据格式回顾（File Format Review）

Python 可处理多种文件类型：

1 文本文件（Text Files）

```
with open("notes.txt", "r", encoding="utf-8") as f:
    content = f.read()
```

2 CSV 文件（Comma-Separated Values）

用于表格数据（如 Excel 导出的数据）。

```
import csv

with open("data.csv", newline='', encoding='utf-8') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

写入 CSV：

```
with open("data.csv", "w", newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Score"])
    writer.writerow(["Alice", 88])
```

---

3 JSON 文件（JavaScript Object Notation）

JSON 是最通用的轻量级数据格式之一，用于数据交换（data exchange）。

```python
import json

data = {"name": "Alice", "age": 25, "skills": ["Python", "SQL"]}

# 写入 JSON 文件
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=4)

# 读取 JSON 文件
with open("data.json", "r", encoding="utf-8") as f:
    info = json.load(f)
    print(info["skills"])
```

9.4 数据库存储（Databases）

1 什么是数据库（What is a Database）

数据库是有组织的数据集合（organized collection of data），
通过结构化查询语言（SQL）进行操作。

2 常见数据库类型

类型	名称	特点
关系型数据库 (RDBMS)	SQLite, MySQL, PostgreSQL	以表格形式存储，支持 SQL
非关系型数据库 (NoSQL)	MongoDB, Redis	文档型、键值型等存储方式

9.5 SQLite 简介（Introduction to SQLite）

SQLite 是 Python 内置支持的轻量级关系数据库（embedded relational database）。
无需安装服务器，只需 `.db` 文件即可使用。

9.6 SQLite 基本操作（Using `sqlite3` Module）

连接数据库

```python
import sqlite3

# 若文件不存在，则自动创建
conn = sqlite3.connect("students.db")
print("Opened database successfully!")
```

创建表（Create Table）

```
cursor = conn.cursor()

cursor.execute('''CREATE TABLE IF NOT EXISTS students (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT,
                    age INTEGER,
                    grade REAL
                )''')

conn.commit()
print("Table created successfully!")
```

插入数据（Insert Data）

```
cursor.execute("INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
               ("Alice", 20, 88.5))
conn.commit()
```

查询数据（Select Data）

```
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

更新与删除（Update & Delete）

```
cursor.execute("UPDATE students SET grade = 92 WHERE name = 'Alice'")
cursor.execute("DELETE FROM students WHERE name = 'Alice'")
conn.commit()
```

关闭连接（Close Connection）

```
conn.close()
```

9.7 使用参数化查询（Parameterized Queries）

为防止 **SQL 注入（SQL Injection）**，要使用 `?` 占位符。

```
name = input("Enter name: ")
cursor.execute("SELECT * FROM students WHERE name = ?", (name,))
```

9.8 事务（Transactions）

SQLite 默认支持事务管理，即一组操作要么全部执行，要么全部失败。

```
try:
    cursor.execute("UPDATE students SET grade=100 WHERE id=1")
    conn.commit()
except:
    conn.rollback()     # 发生错误则回滚
```

9.9 上下文管理简化操作（Using `with` Statement）

可以用 `with` 自动关闭连接：

```
import sqlite3

with sqlite3.connect("students.db") as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM students")
    print(cursor.fetchall())
```

9.10 将数据库结果保存为 JSON 文件

```
import sqlite3, json

with sqlite3.connect("students.db") as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM students")
    data = cursor.fetchall()

# 转为可序列化的结构
students = [{"id": r[0], "name": r[1], "age": r[2], "grade": r[3]} for r in data]

with open("students.json", "w", encoding="utf-8") as f:
    json.dump(students, f, ensure_ascii=False, indent=4)
```

9.11 综合案例：学生管理系统（Student Management System）

实现一个简易的数据库应用：

- 添加学生；

- 查询所有学生；
- 保存到文件。

```
import sqlite3

def create_table():
    with sqlite3.connect("school.db") as conn:
        conn.execute('''CREATE TABLE IF NOT EXISTS students
                        (id INTEGER PRIMARY KEY AUTOINCREMENT,
                         name TEXT, age INTEGER, grade REAL)''')

def add_student(name, age, grade):
    with sqlite3.connect("school.db") as conn:
        conn.execute("INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
                     (name, age, grade))
        print("Added student:", name)

def list_students():
    with sqlite3.connect("school.db") as conn:
        for row in conn.execute("SELECT * FROM students"):
            print(row)

# 测试
create_table()
add_student("Alice", 21, 90)
add_student("Bob", 19, 85)
list_students()
```

9.12 导出为 CSV 文件（Export as CSV）

```
import sqlite3, csv

with sqlite3.connect("school.db") as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM students")
    rows = cursor.fetchall()

with open("students.csv", "w", newline='', encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["ID", "Name", "Age", "Grade"])
    writer.writerows(rows)
```

9.13 使用 Pandas 访问数据库（Using Pandas with Databases）

如果安装了 `pandas`，可以更方便地操作 SQLite。

```python
import sqlite3
import pandas as pd

conn = sqlite3.connect("school.db")
df = pd.read_sql_query("SELECT * FROM students", conn)
print(df)
```

9.14 小结（Summary）

概念	英文 (English)	说明
文件	File	用于简单存储数据
数据库	Database	结构化数据存储
SQLite	SQLite	轻量级嵌入式数据库
SQL	Structured Query Language	数据库查询语言
`sqlite3`	模块	操作 SQLite 数据库
参数化查询	Parameterized Query	防止 SQL 注入
事务	Transaction	保证数据一致性
JSON / CSV	文件格式	常见数据交换格式

第 10 章 图形用户界面编程

(Chapter 10: Graphical User Interface Programming - GUI Programming)

10.1 概述（Overview）

图形用户界面（Graphical User Interface, GUI）
用于让用户通过可视化方式（如按钮、文本框、菜单等）与程序交互。

Python 内置的 GUI 开发工具是 **Tkinter**（读作 *Tee-Kay-Inter*）。

优点：

- 内置模块，无需额外安装；
- 跨平台（Windows/Mac/Linux均支持）；
- 适合中小型桌面应用。

10.2 Tkinter 简介（About Tkinter）

概念	英文	说明
窗口	Window	GUI 的最外层容器
控件	Widget	界面元素（按钮、文本框等）
布局管理器	Layout Manager	控制控件位置的系统
事件	Event	用户操作（点击、输入等）
回调函数	Callback Function	响应事件的函数

导入 Tkinter：

```
import tkinter as tk
```

10.3 创建基本窗口（Creating a Basic Window）

```
import tkinter as tk

root = tk.Tk()                          # 创建主窗口
root.title("My First GUI")              # 设置窗口标题
root.geometry("300x200")                # 设置窗口大小（宽x高）

root.mainloop()                         # 进入消息循环
```

运行后出现一个简单空白窗口。

10.4 添加控件（Adding Widgets）

Tkinter 提供多种控件（widgets）：

控件	英文名称	功能
`Label`	标签	显示文本
`Button`	按钮	点击触发动作
`Entry`	输入框	获取单行输入
`Text`	文本域	多行文本显示
`Frame`	框架	容器控件
`Checkbutton`	复选框	多选开关
`Radiobutton`	单选按钮	互斥选项

控件	英文名称	功能
`Listbox`	列表框	显示列表
`Canvas`	画布	绘图区域

示例：添加标签与按钮

```
import tkinter as tk

def say_hello():
    label.config(text="Hello, Tkinter!")

root = tk.Tk()
root.title("Greeting Example")

label = tk.Label(root, text="点击按钮开始！")
label.pack(pady=10)

button = tk.Button(root, text="Say Hello", command=say_hello)
button.pack()

root.mainloop()
```

10.5 布局管理（Layout Management）

Tkinter 有三种布局管理方式：

管理器	英文	功能
`pack()`	Pack layout	按顺序自动排列
`grid()`	Grid layout	用行列方式排版
`place()`	Absolute layout	按坐标精确定位

例：使用 `grid()` 实现登录表单

```
import tkinter as tk

root = tk.Tk()
root.title("Login Form")

tk.Label(root, text="Username:").grid(row=0, column=0, padx=5, pady=5)
tk.Label(root, text="Password:").grid(row=1, column=0, padx=5, pady=5)

entry_user = tk.Entry(root)
```

```
entry_pass = tk.Entry(root, show="*")

entry_user.grid(row=0, column=1)
entry_pass.grid(row=1, column=1)

def login():
    print("Username:", entry_user.get())
    print("Password:", entry_pass.get())

tk.Button(root, text="Login", command=login).grid(row=2, column=0, columnspan=2, pady=10)

root.mainloop()
```

10.6 文本输入与输出（Text and Input Widgets）

获取文本框输入

```
name = entry_user.get()
```

设置标签文字

```
label.config(text="新的文本")
```

清空输入框

```
entry_user.delete(0, tk.END)
```

10.7 复选框与单选框（Checkbutton & Radiobutton）

```
import tkinter as tk

root = tk.Tk()

agree = tk.BooleanVar()
tk.Checkbutton(root, text="I agree", variable=agree).pack()

choice = tk.StringVar(value="A")
tk.Radiobutton(root, text="Option A", variable=choice, value="A").pack()
tk.Radiobutton(root, text="Option B", variable=choice, value="B").pack()

def show():
    print("Agree:", agree.get())
    print("Choice:", choice.get())

tk.Button(root, text="Submit", command=show).pack()
root.mainloop()
```

10.8 列表框（Listbox）与滚动条（Scrollbar）

```
import tkinter as tk

root = tk.Tk()
scrollbar = tk.Scrollbar(root)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

listbox = tk.Listbox(root, yscrollcommand=scrollbar.set)
for i in range(1, 51):
    listbox.insert(tk.END, f"Item {i}")
listbox.pack(side=tk.LEFT, fill=tk.BOTH)

scrollbar.config(command=listbox.yview)
root.mainloop()
```

10.9 弹出对话框（Messagebox）

```
import tkinter as tk
from tkinter import messagebox

def confirm_exit():
    if messagebox.askyesno("Exit", "Are you sure to quit?"):
        root.destroy()

root = tk.Tk()
tk.Button(root, text="Exit", command=confirm_exit).pack()
root.mainloop()
```

10.10 菜单（Menus）

```
import tkinter as tk

def about():
    tk.messagebox.showinfo("About", "This is a sample GUI app.")

root = tk.Tk()
menu_bar = tk.Menu(root)

file_menu = tk.Menu(menu_bar, tearoff=0)
file_menu.add_command(label="New")
file_menu.add_command(label="Exit", command=root.quit)

help_menu = tk.Menu(menu_bar, tearoff=0)
help_menu.add_command(label="About", command=about)

menu_bar.add_cascade(label="File", menu=file_menu)
menu_bar.add_cascade(label="Help", menu=help_menu)
```

```
root.config(menu=menu_bar)
root.mainloop()
```

10.11 画布（Canvas）绘图

```
import tkinter as tk

root = tk.Tk()
canvas = tk.Canvas(root, width=300, height=200, bg="white")
canvas.pack()

canvas.create_line(0, 0, 200, 100, fill="blue")
canvas.create_rectangle(50, 50, 150, 100, fill="yellow")
canvas.create_oval(100, 50, 180, 130, fill="red")

root.mainloop()
```

10.12 文件选择对话框（File Dialog）

```
from tkinter import filedialog, Tk

root = Tk()
root.withdraw()  # 隐藏主窗口

file_path = filedialog.askopenfilename(title="Select a file")
print("Selected:", file_path)
```

10.13 事件绑定（Event Binding）

可以将函数与用户动作（事件）绑定：

```
import tkinter as tk

def on_key_press(event):
    print(f"Key pressed: {event.keysym}")

root = tk.Tk()
root.bind("<KeyPress>", on_key_press)
root.mainloop()
```

常用事件：

事件	含义
`<Button-1>`	左键点击

事件	含义
`<Double-Button-1>`	双击
`<KeyPress>`	键盘按下
`<Enter>`	鼠标进入控件区域
`<Leave>`	鼠标离开控件

10.14 案例：简单计算器（Simple Calculator）

```
import tkinter as tk

def calculate():
    try:
        result = eval(entry.get())
        label_result.config(text=f"Result: {result}")
    except:
        label_result.config(text="Error")

root = tk.Tk()
root.title("Simple Calculator")

entry = tk.Entry(root, width=20)
entry.pack(pady=5)

tk.Button(root, text="Calculate", command=calculate).pack(pady=5)
label_result = tk.Label(root, text="Result:")
label_result.pack()

root.mainloop()
```

10.15 综合案例：学生信息录入系统（Student Info Form）

```
import tkinter as tk
from tkinter import messagebox

def submit():
    name = entry_name.get()
    age = entry_age.get()
    if not name or not age:
        messagebox.showwarning("Warning", "Please enter all fields!")
    else:
        messagebox.showinfo("Submitted", f"Name: {name}\nAge: {age}")

root = tk.Tk()
root.title("Student Info Form")
```

```
tk.Label(root, text="Name:").grid(row=0, column=0)
entry_name = tk.Entry(root)
entry_name.grid(row=0, column=1)

tk.Label(root, text="Age:").grid(row=1, column=0)
entry_age = tk.Entry(root)
entry_age.grid(row=1, column=1)

tk.Button(root, text="Submit", command=submit).grid(row=2, column=0, columnspan=2, pady=10)
root.mainloop()
```

10.16 小结（Summary）

概念	英文	含义
Tkinter	Tkinter	Python 标准 GUI 库
Widget	控件	界面元素（按钮、文本框等）
Layout	布局管理	控制界面结构
Event	事件	用户行为，如点击或输入
Callback	回调函数	响应事件的函数
Canvas	画布	绘图区域
Menu	菜单	顶部菜单系统
messagebox	消息框	弹出提示或确认
filedialog	文件对话框	打开/保存文件路径
mainloop()	主循环	GUI 程序事件循环

第 11 章：函数式与高级特性（Functional and Advanced Features）

11.1 高阶函数（Higher-order Functions）

定义（Definition）

高阶函数是指**以函数作为参数**或**返回值为函数**的函数。
常见的内置高阶函数包括 `map()`，`filter()`，`reduce()`。

---

1. map() 函数

作用（Purpose）：对可迭代对象（Iterable）中的每个元素执行相同的函数操作，返回新的迭代器。

语法（Syntax）：

```
map(function, iterable)
```

示例（Example）：

```
nums = [1, 2, 3, 4]
result = map(lambda x: x ** 2, nums)
print(list(result))  # 输出 [1, 4, 9, 16]
```

---

2. filter() 函数

作用（Purpose）：根据条件函数（返回 True/False）过滤可迭代对象的元素。

语法（Syntax）：

```
filter(function, iterable)
```

示例（Example）：

```
nums = [1, 2, 3, 4, 5, 6]
even = filter(lambda x: x % 2 == 0, nums)
print(list(even))  # 输出 [2, 4, 6]
```

---

3. reduce() 函数

作用（Purpose）：对序列进行连续累计运算，由 `functools.reduce()` 提供。

语法（Syntax）：

```
from functools import reduce
reduce(function, iterable, initializer(optional))
```

示例（Example）：

```
from functools import reduce
nums = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, nums)
print(total)  # 输出 10
```

说明（Explanation）：

- `reduce()` 每次取两个元素应用函数，将结果继续与下一个元素组合；
- 常用于**累计计算、乘积、聚合等场景**。

11.2 生成器与惰性计算（Generators and Lazy Evaluation）

定义（Definition）

生成器（Generator）是**在函数中使用 `yield` 关键字**定义的可迭代对象，用于**惰性计算（Lazy Evaluation）**。

惰性计算意味着数据按需生成，不会一次性创建全部结果，有效节省内存。

示例（Example）：

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for value in countdown(3):
    print(value)
```

输出：

```
3
2
1
```

特性（Characteristics）：

- `yield` 暂停函数执行并返回中间结果；
- 下一次迭代从 `yield` 后继续；
- 生成器只可迭代一次。

典型应用（Common Use Case）

- 大文件逐行处理；
- 无限序列（如 Fibonacci 数列）；
- 数据流式计算。

11.3 迭代器协议（Iterator Protocol）

概念（Concept）

迭代器（Iterator）是**实现了 `__iter__()` 和 `__next__()` 方法**的对象。
它支持逐步访问数据，不用一次加载全部内容。

示例（Example）：

```python
class Counter:
    def __init__(self, low, high):
        self.current = low
        self.high = high

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.high:
            raise StopIteration
        value = self.current
        self.current += 1
        return value

for c in Counter(1, 3):
    print(c)
```

输出：

```
1
2
3
```

11.4 闭包内部结构解析（Closure Internal Structure）

定义（Definition）

闭包（Closure）是指**内部函数引用外部函数的局部变量**，并在外部函数执行结束后依然保留该变量的状态。

示例（Example）：

```python
def make_multiplier(factor):
    def multiply(x):
        return x * factor
    return multiply

times3 = make_multiplier(3)
print(times3(10))  # 输出 30
```

解析（Explanation）：

- 内部函数 `multiply()` 引用外部作用域的 `factor`；
- `factor` 存储在 `__closure__` 属性中。

查看闭包变量（Inspect Closure）

```python
print(times3.__closure__[0].cell_contents)  # 输出 3
```

闭包能用来实现**数据封装（Data Encapsulation）与装饰器基础逻辑。**

11.5 装饰器应用与封装模式（Decorator Application and Encapsulation Patterns）

概念（Concept）

装饰器（Decorator）是一个用于**动态修改函数或类行为**的可调用对象，核心基于闭包原理。

函数型装饰器（Function Decorator）示例：

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} finished")
        return result
    return wrapper

@log_call
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
```

输出：

```
Calling greet
Hello, Alice!
greet finished
```

应用场景（Use Cases）

- 函数性能计时（Performance Timing）
- 权限验证（Access Control）
- 日志记录与追踪（Logging & Tracing）

11.6 内省与动态类型特性（Introspection and Dynamic Typing Features）

定义（Definition）

内省（Introspection）是 Python 的运行时特性，允许在程序执行期间访问对象的类型信息与属性结构。

常用函数（Common Built-ins）：

函数	功能说明（Description）
`getattr(obj, name[, default])`	获取对象属性
`setattr(obj, name, value)`	动态设置属性
`hasattr(obj, name)`	判断对象是否具有属性
`delattr(obj, name)`	删除对象属性
`type(obj)`	返回对象类型
`dir(obj)`	列出对象成员

示例（Example）：

```python
class Person:
    pass

p = Person()
setattr(p, "name", "Alice")
print(getattr(p, "name"))  # 输出 Alice

if hasattr(p, "name"):
    print("Attribute exists")
```

动态特性应用（Dynamic Typing in Practice）

- 反射模块加载（Dynamic Module Loading）；
- ORM 框架动态绑定字段；
- 动态创建类与方法。

第 12 章：模块、包与环境（Modules, Packages and Environments）

12.1 模块导入机制（Module Import Mechanism）

定义（Definition）

模块（Module）是一个包含 Python 代码的文件，用于组织函数、类和变量以提高代码可维护性与复用性。
Python 使用 `import` 语句引入模块。

导入方式（Import Methods）

1. **直接导入（Direct Import）**

```
import math
print(math.sqrt(16))  # 输出 4.0
```

2. **选择性导入（Selective Import）**

```
from math import sqrt, pi
print(sqrt(9), pi)
```

3. **模块别名（Module Alias）**

```
import numpy as np
```

4. **动态导入（Dynamic Import）**

```
mod = __import__('math')
print(mod.factorial(5))  # 输出 120
```

模块搜索路径（Module Search Path）

Python 导入模块时按以下顺序搜索：

1. 当前工作目录（Current Directory）
2. 环境变量 `PYTHONPATH` 指定的目录
3. 标准库（Standard Library）目录
4. 第三方包（Third-party Packages）目录

可通过查看：

```
import sys
print(sys.path)
```

获取模块搜索路径。

12.2 自定义包与目录结构（Custom Packages and Directory Structure）

定义（Definition）

包（Package）是包含 `__init__.py` 文件的目录，用于组织多个模块。
使 Python 将该目录视为一个逻辑命名空间。

包结构（Package Structure）示例：

```
project/
|
├── main.py
└── utils/
    ├── __init__.py
    ├── file_ops.py
    └── string_ops.py
```

在 main.py 中导入方式：

```python
from utils import file_ops
file_ops.read_file("data.txt")
```

`__init__.py` 的作用

- 标识目录为包；
- 可定义包级变量或自动加载模块。

示例：

```python
# utils/__init__.py
print("utils package loaded")
```

12.3 虚拟环境（Virtual Environments）

定义（Definition）

虚拟环境（Virtual Environment）用于为不同项目创建相互独立的 Python 包依赖空间，避免版本冲突。

---

1. venv 虚拟环境

（Python 内置方式）

创建环境：

```
python -m venv venv_name
```

激活环境：

- Windows:

  ```
  venv_name\Scripts\activate
  ```

- macOS / Linux:

  ```
  source venv_name/bin/activate
  ```

退出环境：

```
deactivate
```

---

2. conda 环境

（适用于数据科学或复杂依赖场景）

常用命令：

```
conda create -n myenv python=3.11
conda activate myenv
conda install numpy pandas
conda deactivate
```

---

3. 环境依赖管理（Dependency Management）

导出依赖：

```
pip freeze > requirements.txt
```

安装依赖：

```
pip install -r requirements.txt
```

12.4 第三方包管理（Third-party Package Management with pip）

pip 常用命令（Common pip Commands）

命令	功能说明（Description）
`pip install package_name`	安装包
`pip uninstall package_name`	卸载包
`pip list`	查看已安装包
`pip show package_name`	查看包信息
`pip install -U package_name`	升级包
`pip install package==version`	安装指定版本

示例：

```
pip install requests
```

在代码中使用：

```
import requests
resp = requests.get("https://api.github.com")
print(resp.status_code)
```

离线与镜像源（Offline & Mirror Sources）

国内可使用镜像源：

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple numpy
```

12.5 模块缓存与命名空间原理（Module Caching and Namespace Principles）

模块缓存机制（Module Caching）

- Python 使用 `sys.modules` 缓存已加载模块，减少重复导入。
- 修改模块后需使用 `importlib.reload()` 重新加载。

```
import sys, importlib, mymodule
importlib.reload(mymodule)
```

命名空间原理（Namespace Principle）

命名空间（Namespace）用于隔离变量名称，避免冲突。
主要分为：

- 局部命名空间（Local Namespace）
- 全局命名空间（Global Namespace）
- 内建命名空间（Built-in Namespace）

优先级顺序（LEGB Rule）：
Local → Enclosing → Global → Built-in

示例（Example）：

```
x = 10

def outer():
    x = 20
    def inner():
        x = 30
        print(x)
    inner()

outer()  # 输出 30
```

解释：`inner()` 的 `x` 在局部命名空间中优先级最高。

第 13 章：调试与测试（Debugging and Testing）

13.1 调试流程与工具（Debug Workflow and Tools）

定义（Definition）

调试（Debugging）是发现并修复程序逻辑或运行错误的过程。
在 Python 中，调试可通过简单的打印输出、命令行调试器或集成开发环境（IDE）来进行。

常见错误类型扩展表（Extended Table of Common Python Error Types

异常类型（Exception Type）	英文名称（English Name）	触发场景（Typical Cause）	示例（Example）	说明（Explanation）
SyntaxError	Syntax Error	Python 语法结构不合法	`if x = 5:`	编译阶段解析错误

异常类型（Exception Type）	英文名称（English Name）	触发场景（Typical Cause）	示例（Example）	说明（Explanation）
IndentationError	Indentation Error	缩进不一致或缺失	`if True:\nprint("Hi")`	Python 对缩进严格要求
NameError	Name Error	使用未定义变量	`print(x)`	变量未声明或超出作用域
TypeError	Type Error	不兼容类型操作	`'3' + 3`	类型冲突导致运行错误
ValueError	Value Error	值不符合预期格式	`int('abc')`	内容无法转换为目标类型
IndexError	Index Error	列表或元组索引超界	`lst = [1]; lst[2]`	下标越界访问
KeyError	Key Error	字典键不存在	`d = {}; d['missing']`	访问未定义键值
AttributeError	Attribute Error	访问不存在的对象属性	`'abc'.append(1)`	对象无此方法或属性
ImportError / **ModuleNotFoundError**	Import Error / Module Not Found Error	模块导入失败	`import nonexist`	模块在路径中未找到
ZeroDivisionError	Zero Division Error	被除数为零	`10 / 0`	算术除法错误
IOError / **OSError**	Input/Output Error / OS Error	文件或系统操作失败	`open('missing.txt')`	文件路径或权限问题
FileNotFoundError	File Not Found Error	指定文件不存在	`open('nofile.txt')`	`IOError` 的子类
PermissionError	Permission Error	无访问权限	`open('/root/secret.txt')`	用户权限不够导致
RuntimeError	Runtime Error	通用运行时错误	`raise RuntimeError("Error")`	非特定错误类型的通用类别
RecursionError	Recursion Error	递归调用过深超出限制	无限递归函数调用	默认递归限制为 1000 层
MemoryError	Memory Error	内存不足	大型数组创建失败	无法分配新对象内存
OverflowError	Overflow Error	数值运算结果超出允许范围	`math.exp(1000)`	数学结果超过浮点表示范围
StopIteration	Stop Iteration	迭代器手动终止	`next(it)` 到末尾	用于 `for` 循环内部机制
AssertionError	Assertion Error	`assert` 断言失败	`assert False, "fail"`	测试逻辑假设条件失败
ArithmeticError	Arithmetic Error	算术相关错误的总类	其子类包括 `ZeroDivisionError` 等	所有数学计算错误基础类
EOFError	End of File Error	输入流无内容可读	`input()` 遇意外 EOF	常见于文件/交互输入
NotImplementedError	Not Implemented Error	抽象方法未实现	`raise NotImplementedError()`	用于定义接口骨架
AttributeError	Attribute Error	对象访问不存在属性	`'str'.pop()`	缺少属性或方法
UnicodeError	Unicode Error	字符编码/解码失败	`'你好'.encode('ascii')`	编解码器不匹配

异常类型（Exception Type）	英文名称（English Name）	触发场景（Typical Cause）	示例（Example）	说明（Explanation）
DeprecationWarning (警告类)	Deprecation Warning	使用已弃用功能	调用旧 API	不阻断程序运行，仅警示
Warning	Warning	一般性警告信息	`import warnings; warnings.warn("注意")`	可被捕获但不会中断执行

按类别层次分类（Categorization by Class Hierarchy）

Python 异常类结构可总结为如下层次：

```
BaseException
├── SystemExit
├── KeyboardInterrupt
├── GeneratorExit
└── Exception
    ├── ArithmeticError
    |   ├── ZeroDivisionError
    |   ├── OverflowError
    |   └── FloatingPointError
    ├── AssertionError
    ├── AttributeError
    ├── EOFError
    ├── ImportError
    ├── LookupError
    |   ├── IndexError
    |   └── KeyError
    ├── MemoryError
    ├── NameError
    |   └── UnboundLocalError
    ├── OSError
    |   ├── FileNotFoundError
    |   ├── PermissionError
    |   └── TimeoutError
    ├── RuntimeError
    |   └── RecursionError
    ├── SyntaxError
    |   └── IndentationError
    ├── TypeError
    ├── ValueError
    |   └── UnicodeError
    └── Warning（非中断型警告）
```

示例：如何捕获常见异常（Example: Catching Common Exceptions）

```
try:
    num = int(input("请输入数字: "))
    print(10 / num)
except ValueError:
    print("输入的不是合法数字")
except ZeroDivisionError:
    print("除数不能为零")
except Exception as e:
    print("发生其他错误:", type(e).__name__)
```

IDE 调试（Visual Studio Code / PyCharm）

调试流程通用步骤：

1. 设置断点（Breakpoint）
2. 启动调试器（Start Debugger）
3. 单步执行（Step Over / Step Into）
4. 监视变量（Watch Variables）
5. 查看调用栈（Call Stack）

13.2 打印调试与断点调试（Print Debugging and Breakpoint Debugging）

打印调试（Print Debugging）

最常用且快速的手段，通过打印变量状态帮助定位问题。

示例：

```
def compute_sum(n):
    total = 0
    for i in range(n):
        print("迭代:", i, "当前和:", total)
        total += i
    return total
```

适用于：

- 快速理解程序流程；
- 小型、无性能要求脚本。

不足：

- 修改代码后需还原；
- 不便于复杂逻辑追踪。

断点调试（Breakpoint Debugging）

Python 提供内置模块 `pdb`（Python Debugger）。

示例：

```
import pdb

def division(a, b):
    pdb.set_trace()
    return a / b

division(10, 0)
```

常用命令（pdb Commands）：

命令	含义（Meaning）
`n`	执行下一行（Next）
`s`	进入函数（Step Into）
`c`	继续执行到下一个断点
`q`	退出调试器（Quit）
`p <var>`	打印变量值（Print variable）

13.3 assert 语句与防御性编程（Assertions and Defensive Programming）

定义（Definition）

`assert` 是一种快速检测假设是否成立的语句，用于早期发现逻辑错误。
格式：

```
assert condition, message
```

示例：

```
def divide(a, b):
    assert b != 0, "除数不能为零"
    return a / b
```

执行中若 `b == 0`，程序会抛出：

```
AssertionError: 除数不能为零
```

防御性编程（Defensive Programming）

在关键逻辑处主动检查前置条件（Preconditions），保证稳定性。

示例：

```
def process_student(data):
    if not isinstance(data, dict):
        raise TypeError("输入必须为字典类型")
```

13.4 单元测试基础（Unit Testing Basics）

定义（Definition）

单元测试（Unit Testing）用于独立验证程序中最小功能单元（如函数或方法）的正确性。

Python 提供内置模块 `unittest`。

---

unittest 示例（Example）

文件结构：

```
project/
|
├── calculator.py
└── test_calculator.py
```

calculator.py

```
def add(a, b):
    return a + b
```

test_calculator.py

```
import unittest
from calculator import add

class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertNotEqual(add(1, 1), 3)

if __name__ == "__main__":
    unittest.main()
```

运行：

```
python -m unittest test_calculator.py
```

输出：

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s
OK
```

常用断言方法（Common Assertions）

方法	功能
`assertEqual(a, b)`	是否相等
`assertNotEqual(a, b)`	是否不相等
`assertTrue(x)`	是否为真
`assertFalse(x)`	是否为假
`assertIn(a, b)`	元素是否在容器中
`assertRaises(Exception, func)`	是否抛出异常

---

pytest 框架

`pytest` 是更简洁、现代的测试框架。

安装并运行：

```
pip install pytest
pytest
```

测试函数格式：

```
def test_addition():
    assert 1 + 1 == 2
```

pytest 自动发现以 “test_” 开头的函数进行测试。

13.5 日志记录与错误追踪（Logging and Error Tracking）

定义（Definition）

日志（Logging）用于在程序运行时记录系统行为、状态变化和错误信息，便于追踪与诊断。

基础示例（Basic Example）

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("程序启动")
logging.warning("磁盘空间不足")
logging.error("文件读取失败")
```

输出格式：

```
2026-04-28 15:30:12 - INFO - 程序启动
2026-04-28 15:30:12 - ERROR - 文件读取失败
```

日志级别（Logging Levels）

级别名称	英文名称	用途
DEBUG	调试信息	最详细的程序内部状态
INFO	一般运行信息	普通操作消息
WARNING	警告	可能的问题或风险
ERROR	错误	不影响继续运行的错误
CRITICAL	严重错误	程序可能终止

日志输出到文件（Output to File）

```
logging.basicConfig(
    filename="app.log",
    filemode="w",
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
```

运行后日志会写入 `app.log` 文件中。

错误追踪（Error Tracking with Traceback）

`traceback` 模块可用于获取异常的详细堆栈信息。

示例：

```
import traceback

try:
    1 / 0
except Exception as e:
    print("错误信息:")
    traceback.print_exc()
```

第 14 章：进阶主题（Advanced Topics）

14.1 生成器与迭代器（Generators and Iterators）

定义（Definition）

生成器（Generator）与迭代器（Iterator）是 Python 支持惰性求值（Lazy Evaluation）的核心机制，用于节省内存并提升效率。

迭代器（Iterator）

迭代器是实现 `__iter__()` 与 `__next__()` 方法的对象。
调用 `next()` 可逐步获取元素，直到触发 `StopIteration`。

示例：

```
nums = [1, 2, 3]
it = iter(nums)
print(next(it))  # 输出 1
print(next(it))  # 输出 2
```

生成器（Generator）

生成器是一种特殊的迭代器，通过函数中的 `yield` 关键字定义。

示例：

```python
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for i in countdown(3):
    print(i)
```

输出：

```
3
2
1
```

生成器表达式（Generator Expression）

形式类似列表推导式，但使用圆括号并惰性计算。

```python
gen = (x**2 for x in range(5))
print(next(gen))  # 输出 0
print(next(gen))  # 输出 1
```

优点（Advantages）

- 占用内存少；
- 可逐步处理大型数据流；
- 可与迭代工具（如 `itertools`）结合。

14.2 装饰器（Decorators）

定义（Definition）

装饰器用于在不改变原函数代码的情况下，为函数或类添加额外功能。
其本质是一个返回函数的高阶函数。

基本语法（Basic Syntax）

```python
def decorator(func):
    def wrapper():
        print("执行前")
        func()
        print("执行后")
    return wrapper

@decorator
def greet():
    print("你好")

greet()
```

输出：

```
执行前
你好
执行后
```

带参数的装饰器（Decorators with Arguments）

```python
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def hello():
    print("Hello!")

hello()
```

输出：

```
Hello!
Hello!
Hello!
```

常见用途（Common Use Cases）

类型	应用场景
日志记录	函数执行前后记录日志
性能监测	计算运行时间
权限验证	控制接口访问权限
缓存结果	提高重复计算效率

14.3 匿名函数与函数式编程（Lambda and Functional Programming）

匿名函数（Lambda Expressions）

用于定义简短的临时函数。

```
square = lambda x: x ** 2
print(square(5))
```

等效于：

```
def square(x):
    return x ** 2
```

函数式工具（Functional Tools）

1. `map()`

把函数应用到序列的每个元素。

```
nums = [1, 2, 3]
print(list(map(lambda x: x * 2, nums)))
```

2. `filter()`

筛选满足条件的元素。

```
nums = [1, 2, 3, 4]
print(list(filter(lambda x: x % 2 == 0, nums)))
```

3. `reduce()`

通过累积计算合并序列元素。

```
from functools import reduce
nums = [1, 2, 3, 4]
print(reduce(lambda a, b: a + b, nums))
```

14.4 迭代工具与生成器模块（Itertools and Generator Utilities）

itertools 模块简介

`itertools` 提供高性能迭代器构造函数。

函数	作用	示例
`count(start, step)`	无限计数生成器	`itertools.count(1)`
`cycle(iterable)`	无限循环序列	`itertools.cycle([1, 2])`
`repeat(obj, n)`	重复对象 n 次	`itertools.repeat("A", 3)`
`chain()`	拼接多个可迭代对象	`itertools.chain([1], [2, 3])`
`combinations()`	生成组合	`itertools.combinations("ABCD", 2)`
`permutations()`	生成排列	`itertools.permutations([1, 2, 3])`

示例：组合生成

```
import itertools
for combo in itertools.combinations('ABC', 2):
    print(combo)
```

输出：

```
('A', 'B')
('A', 'C')
('B', 'C')
```

14.5 上下文管理器（Context Managers）

定义（Definition）

上下文管理器用于在代码块前后自动执行预定义操作（例如资源释放）。
通过 `with` 语句实现。

示例

```
with open('data.txt', 'w') as f:
    f.write("Hello World")
```

当退出 `with` 块时，文件会自动关闭。

自定义上下文管理器

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        print("运行时间:", time.time() - self.start)

with Timer():
    sum(range(1000000))
```

14.6 元类与动态类创建（Metaclasses and Dynamic Class Creation）

定义（Definition）

元类（Metaclass）是 “创建类的类”。
类定义时实际上是由其元类生成的对象。

基本示例

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        print("正在创建类:", name)
        return super().__new__(cls, name, bases, dct)

class Example(metaclass=Meta):
    pass
```

输出：

```
正在创建类: Example
```

用途总结

- 在类创建阶段动态修改属性；
- 自动注册类；
- 实现框架级约束（如 Django ORM 元类）。

14.7 协程与异步编程（Coroutines and Async Programming）

定义（Definition）

协程（Coroutine）是可暂停和恢复执行的函数，是异步编程的基础。

关键字：`async`，`await`

示例：异步函数（Async Function）

```
import asyncio

async def task(name):
    print(f"{name} 开始")
    await asyncio.sleep(1)
    print(f"{name} 结束")

async def main():
    await asyncio.gather(task("A"), task("B"))

asyncio.run(main())
```

输出：

```
A 开始
B 开始
A 结束
B 结束
```

应用场景（Applications）

- 网络请求同时处理（如爬虫系统）
- 高并发任务调度
- 异步 I/O （文件或数据库操作）

14.8 内省与反射（Introspection and Reflection）

定义（Definition）

内省（Introspection）用于在运行时查看对象类型、属性或方法。

常用函数

函数	功能说明
`type(obj)`	返回对象类型
`dir(obj)`	列出可访问属性与方法
`getattr(obj, name)`	获取属性值
`setattr(obj, name, value)`	动态设置属性值
`hasattr(obj, name)`	判断属性是否存在

示例：

```python
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Alice")

print(dir(p))
print(getattr(p, "name"))
setattr(p, "age", 23)
print(hasattr(p, "age"))
```

14.9 序列化与反序列化（Serialization and Deserialization）

定义（Definition）

序列化用于将对象转换为可存储或传输的格式（如 JSON、pickle）。

使用 `json` 模块

```
import json
data = {"name": "Tom", "age": 20}
json_str = json.dumps(data)
print(json_str)
loaded = json.loads(json_str)
print(loaded["name"])
```

使用 `pickle` 模块

支持任意 Python 对象的序列化。

```
import pickle
obj = [1, 2, 3]
serialized = pickle.dumps(obj)
restored = pickle.loads(serialized)
print(restored)
```

14.10 正则表达式与文本处理（Regular Expressions and Text Processing）

正则基础（Basics）

```
import re
text = "Email: example@test.com"
match = re.search(r"\w+@\w+\.\w+", text)
if match:
    print(match.group())
```

常见正则模式（Common Patterns）

模式	含义
`\d`	数字
`\w`	字母或数字

模式	含义
`\s`	空白字符
`^`	行首匹配
`$`	行尾匹配
`.`	任意单个字符
`+`	前项至少出现一次
`{n,m}`	前项出现 n 到 m 次

第 15 章：递归函数（Recursive Functions）

15.1 递归的基本概念（Basic Concept of Recursion）

定义（Definition）

递归（Recursion）是一种函数调用自身以实现问题分解的编程技巧。
在递归结构中，一个大问题被分解成若干相似的更小问题。

递归的两个基本条件

1. **基例（Base Case）**：定义终止条件，防止无限递归。
2. **递归步骤（Recursive Step）**：函数调用自身解决子问题。

示例：阶乘（Factorial）

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))
```

输出：

```
120
```

递归过程示意

计算 `factorial(3)`：

```
factorial(3)
= 3 * factorial(2)
= 3 * (2 * factorial(1))
= 3 * (2 * 1)
= 6
```

15.2 递归的调用栈（Call Stack）

每次函数调用，Python 都会将当前函数状态压入**调用栈（Call Stack）**。
当递归结束后，栈逐层弹出以恢复先前的执行状态。

打印递归调用顺序：

```python
def count_down(n):
    print("进入层次:", n)
    if n == 0:
        print("到达终点")
    else:
        count_down(n - 1)
    print("退出层次:", n)

count_down(3)
```

输出：

```
进入层次: 3
进入层次: 2
进入层次: 1
进入层次: 0
到达终点
退出层次: 0
退出层次: 1
退出层次: 2
退出层次: 3
```

15.3 常见递归实例（Typical Recursive Examples）

1. 阶乘（Factorial）

```python
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

2. 斐波那契数列（Fibonacci Sequence）

定义：

```
F(0) = 0
F(1) = 1
F(n) = F(n-1) + F(n-2)
```

实现：

```python
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(6))  # 输出 8
```

---

3. 列表求和（List Sum）

```python
def list_sum(lst):
    if not lst:
        return 0
    return lst[0] + list_sum(lst[1:])

print(list_sum([1, 2, 3, 4]))  # 输出 10
```

---

4. 反向打印字符串（Reverse String）

```python
def reverse(s):
    if s == "":
        return s
    return reverse(s[1:]) + s[0]

print(reverse("python"))  # 输出 "nohtyp"
```

---

5. 目录遍历（Directory Traversal）

```
import os

def scan(path, depth=0):
    for name in os.listdir(path):
        full = os.path.join(path, name)
        print("  " * depth + name)
        if os.path.isdir(full):
            scan(full, depth + 1)

scan(".")
```

15.4 递归与迭代（Recursion vs. Iteration）

特征	递归（Recursion）	迭代（Iteration）
核心机制	函数自调用	循环控制结构
状态存储	系统调用栈	循环变量
优点	结构简洁、逻辑清晰	内存效率高、性能更好
缺点	栈深受限、性能可能较低	不如递归直观
适用场景	分形、树遍历、分治算法	大范围数值计算

15.5 Python 的递归限制（Recursion Limit）

Python 默认的最大递归深度约为 **1000 层**。超出会抛出：

```
RecursionError: maximum recursion depth exceeded
```

查看或修改递归深度限制：

```
import sys
print(sys.getrecursionlimit())
sys.setrecursionlimit(2000)
```

注意：修改递归限制需谨慎，过深递归可能导致栈溢出或程序崩溃。

15.6 尾递归与优化（Tail Recursion and Optimization）

尾递归（Tail Recursion）

尾递归是指函数在**返回前最后一步**只调用自身而不再做其他运算，可在某些语言中被优化以节省栈空间（**尾递归优化**）。

普通递归：

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

尾递归形式：

```
def factorial_tail(n, acc=1):
    if n == 1:
        return acc
    return factorial_tail(n - 1, acc * n)
```

虽然 Python **不支持尾递归优化**，但这种写法逻辑上更清晰，也便于后续转换为循环。

15.7 分治算法与递归（Divide and Conquer Algorithms）

递归常用于**分治（Divide and Conquer）**算法中：
把大问题分成若干个可独立求解的子问题，然后合并结果。

示例：二分查找（Binary Search）

```
def binary_search(lst, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if lst[mid] == target:
        return mid
    elif lst[mid] > target:
        return binary_search(lst, target, low, mid - 1)
    else:
        return binary_search(lst, target, mid + 1, high)

nums = [1, 3, 5, 7, 9]
print(binary_search(nums, 7, 0, len(nums) - 1))  # 输出 3
```

15.8 使用递归构建复杂结构（Recursive Structures）

树的遍历（Tree Traversal）

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def preorder(node):
    if node:
        print(node.val)
        preorder(node.left)
        preorder(node.right)

# 构造简单二叉树
root = Node(1)
root.left = Node(2)
root.right = Node(3)
preorder(root)
```

输出：

```
1
2
3
```

15.9 总结与实践（Summary and Practice）

知识要点总结

- 递归函数必须**包含终止条件**；
- 调用栈控制函数执行顺序；
- 对性能敏感任务应注意栈溢出风险；
- 可尝试将递归转换为循环获得更高效率。

第 16 章：排序算法（Sorting Algorithms）

16.1 排序的基本概念（Basic Concept of Sorting）

定义（Definition）

排序（Sorting）是将一组数据按某种顺序（如升序或降序）排列。
它是算法中最常见、最基础的操作之一。

排序分类（Types of Sorting）

分类方式	类型举例
按策略	比较排序、非比较排序
按内外存	内部排序（数据在内存中）、外部排序（数据过大需存储中间结果）
按算法特性	稳定排序、不稳定排序
按实现方式	迭代算法、递归算法

常见排序算法汇总

算法名称	时间复杂度	空间复杂度	稳定性	递归类型
冒泡排序（Bubble Sort）	$O(n^2)$	O(1)	✅ 稳定	否
选择排序（Selection Sort）	$O(n^2)$	O(1)	❌ 不稳定	否
插入排序（Insertion Sort）	$O(n^2)$	O(1)	✅ 稳定	否
快速排序（Quick Sort）	O(n log n)	O(log n)	❌ 不稳定	✅ 是
归并排序（Merge Sort）	O(n log n)	O(n)	✅ 稳定	✅ 是
堆排序（Heap Sort）	O(n log n)	O(1)	❌ 不稳定	否

16.2 冒泡排序（Bubble Sort）

算法思想

相邻元素两两比较，如果顺序错误则交换，像“气泡”一样大值逐步上浮到末尾。

示例代码

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort([5, 2, 9, 1, 5, 6]))
```

输出：

```
[1, 2, 5, 5, 6, 9]
```

优化版本（提前终止）

```python
def bubble_sort_optimized(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

16.3 选择排序（Selection Sort）

算法思想

每轮选择最小元素放入已排序区开头。

示例代码

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
```

复杂度分析

- 时间：$O(n^2)$
- 空间：$O(1)$
- 稳定性：不稳定（存在交换破坏顺序）

16.4 插入排序（Insertion Sort）

算法思想

将未排序部分的元素逐一插入到已排序部分的正确位置中。

示例代码

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([4, 3, 2, 1]))
```

输出：

```
[1, 2, 3, 4]
```

特点

- 适用于**少量元素或近乎有序列表**；
- 性能在小规模数据时优于某些复杂算法。

16.5 快速排序（Quick Sort）

算法思想

利用“分治”与“递归”：

1. 选择一个基准值（pivot）；
2. 将比 pivot 小的元素放左侧，大的放右侧；
3. 递归排序左右两部分。

代码实现

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

print(quick_sort([3, 6, 8, 10, 1, 2, 1]))
```

输出：

```
[1, 1, 2, 3, 6, 8, 10]
```

特征与优化

特性	说明
平均复杂度	O(n log n)
最坏情况	$O(n^2)$（当数组已接近有序且基准选择不当时）
空间复杂度	O(log n)（递归栈）
是否稳定	否

优化方向：

- 使用**随机基准**或“三数取中”方法；

- 小规模分区可切换为插入排序。

16.6 归并排序（Merge Sort）

算法思想

1. 将列表递归地一分为二；
2. 对左右部分分别排序；
3. 将两个有序子序列合并。

示例实现

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([5, 2, 4, 6, 1, 3]))
```

输出：

```
[1, 2, 3, 4, 5, 6]
```

特点

- 稳定排序；
- 典型的递归分治算法；
- 适合处理大型序列或链表。

16.7 堆排序（Heap Sort）

算法思想

1. 将数组构造成最大堆（父节点大于子节点）；
2. 交换堆顶与末尾元素，将未排序部分继续堆化。

代码示例：

```
def heapify(arr, n, i):
    largest = i
    left, right = 2*i + 1, 2*i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

print(heap_sort([4, 10, 3, 5, 1]))
```

输出：

```
[1, 3, 4, 5, 10]
```

16.8 Python 内置排序（Built-in Sorting）

Python 内置的 `sorted()` 与列表方法 `.sort()` 使用 **Timsort** 算法 ——
综合了 **归并排序** 和 **插入排序** 的优势。

```
nums = [3, 1, 4, 1, 5]
print(sorted(nums))              # 返回新列表
nums.sort(reverse=True)
print(nums)                      # 原地排序
```

输出：

```
[1, 1, 3, 4, 5]
[5, 4, 3, 1, 1]
```

关键参数

参数	说明
`key`	指定排序依据函数（如 `key=len`）
`reverse`	是否降序排序（默认为 `False`）

示例：按字符串长度排序

```python
words = ["python", "c", "java", "rust"]
print(sorted(words, key=len))
```

输出：

```
['c', 'java', 'rust', 'python']
```

16.9 各算法性能对比总结

算法	时间复杂度（平均）	空间复杂度	稳定	适合场景
冒泡排序	$O(n^2)$	O(1)	✅	教学演示
选择排序	$O(n^2)$	O(1)	❌	小规模数据
插入排序	$O(n^2)$	O(1)	✅	近乎有序数组
快速排序	O(n log n)	O(log n)	❌	普通场景，高性能排序
归并排序	O(n log n)	O(n)	✅	稳定性要求高的数据
堆排序	O(n log n)	O(1)	❌	固定资源场景（如嵌入式）
Timsort（内置）	O(n log n)	O(n)	✅	综合性能最优

第 17 章：查找算法（Searching Algorithms）

17.1 查找的基本概念（Basic Concept of Searching）

定义（Definition）

查找（Searching）是指在数据集中定位目标元素的位置或验证其是否存在。
它是数据结构和算法中最核心的操作之一。

查找问题的形式化表达

给定一个元素集合：

```
A = [a1, a2, a3, ..., an]
```

求：

```
index(x)  →  元素 x 在 A 中的下标或 None（若不存在）
```

查找算法的分类

分类标准	类型举例
按数据结构	线性查找、二分查找、哈希查找、树查找
按存储形式	顺序表查找、链表查找、索引查找
按算法特性	静态查找（不更改结构）、动态查找（允许插入删除）

17.2 线性查找（Linear Search）

算法思想

从头到尾顺序扫描整个序列，逐个比较目标值。

代码实现

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

nums = [4, 2, 7, 1, 3]
print(linear_search(nums, 7))
```

输出：

```
2
```

分析

- 平均时间复杂度：O(n)
- 空间复杂度：O(1)
- 适用于：**无序数据、小规模数据**
- 优点：实现简单
- 缺点：效率低

优化方式：提前判断

如果列表已排序，可提前终止：

```python
def ordered_linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
        elif arr[i] > target:
            break
    return -1
```

17.3 二分查找（Binary Search）

算法思想

在**有序序列**中查找：
通过比较中间元素，逐步缩小查找范围，类似折半搜索。

递归实现

```python
def binary_search(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)

nums = [1, 3, 5, 7, 9]
print(binary_search(nums, 7, 0, len(nums) - 1))
```

输出：

```
3
```

迭代实现

```python
def binary_search_iterative(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

分析

项目	说明
时间复杂度	O(log n)
空间复杂度（递归）	O(log n)
适用数据	有序序列
优点	高效、实现简洁
缺点	不适用于无序数据

可视化搜索过程示例

查找 7 于 `[1, 3, 5, 7, 9]`

```
low=0, high=4 → mid=2 → arr[mid]=5 → target>5 → 新区间=[3,4]
low=3, high=4 → mid=3 → arr[mid]=7 → 找到目标
```

17.4 哈希查找（Hash Search）

算法思想

通过**哈希函数（Hash Function）**将键映射为索引位置。
查找过程可在常数时间完成。

示例：Python 字典

```
students = {"Alice": 85, "Bob": 90, "Charlie": 78}
print(students["Bob"])
```

输出：

```
90
```

原理说明

1. 哈希函数计算索引：`index = hash(key) % m`
2. 直接定位元素而非遍历
3. 解决冲突方式：开放地址、链地址、再哈希等

哈希表性能

操作	平均复杂度	最坏复杂度
查找	O(1)	O(n)
插入	O(1)	O(n)
删除	O(1)	O(n)

自定义哈希查找示例

```
def simple_hash(key, size):
    return hash(key) % size

hash_table = [[] for _ in range(5)]

def insert(table, key, value):
    idx = simple_hash(key, len(table))
    table[idx].append((key, value))

def search(table, key):
    idx = simple_hash(key, len(table))
```

```
    for k, v in table[idx]:
        if k == key:
            return v
    return None

insert(hash_table, "apple", 10)
insert(hash_table, "banana", 20)
print(search(hash_table, "banana"))
```

输出：

```
20
```

17.5 查找算法性能对比

算法类型	时间复杂度（平均）	适用数据结构	优点	缺点
线性查找	O(n)	任意列表	简单、普适	慢
二分查找	O(log n)	有序列表	高效	需排序
哈希查找	O(1)	哈希表	最快	需额外空间；需处理冲突

17.6 二分查找的扩展应用（Binary Search Extensions）

1. 查找插入位置

```
def binary_search_insert(arr, target):
    low, high = 0, len(arr)
    while low < high:
        mid = (low + high) // 2
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low

nums = [1, 3, 5, 7]
print(binary_search_insert(nums, 6))
```

输出：

```
3
```

---

2. 查找最左或最右匹配（重复元素情况）

```
def binary_search_left(arr, target):
    low, high = 0, len(arr)
    while low < high:
        mid = (low + high) // 2
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low
```

---

3. 在区间中应用二分

典型场景：求最小满足条件的数值，如数值优化、根查找问题。

17.7 使用 Python 标准库查找

Python 已为常见查找提供高效工具。

1. `in` 运算符

```
nums = [1, 3, 5, 7]
print(5 in nums)
```

输出：

```
True
```

---

2. `index()` 方法

```
nums = [10, 20, 30]
print(nums.index(20))
```

输出：

```
1
```

---

3. `bisect` 模块（二分查找工具）

```
import bisect
arr = [1, 3, 4, 7, 9]
print(bisect.bisect_left(arr, 4))
print(bisect.bisect_right(arr, 4))
```

输出：

```
2
3
```

Python 内置查找的特点

- 对列表、集合、字典提供统一接口；
- 集合和字典基于哈希表，查找复杂度为 O(1)；
- 列表支持顺序查找与 `bisect` 二分查找。

17.8 小结与练习

要点回顾

- 查找算法用于定位数据位置或判断存在性；
- 二分查找和哈希查找是程序效率提升的关键；
- Python 内置查找功能高度优化，但掌握原理有助于理解时间复杂度与设计数据结构。

练习题

1. 编写一个二分查找函数查找重复元素的最右位置；
2. 使用链地址法实现完整的哈希表；
3. 对比线性查找与哈希查找在 10000 个元素上的性能差异；
4. 用递归实现二分查找的“插入位置”功能。

第 18 章：搜索与排序的综合应用

18.1 引言（Introduction）

排序与查找几乎是所有数据处理的核心。
排序（Sorting） 将数据组织为特定顺序，
查找（Searching） 则在有序数据中快速定位。

在实际程序中，它们往往 **配合使用**：

> “先排序，后查找” 是通用优化策略。

18.2 从排序到查找的关系（Relation Between Sorting and Searching）

关系	说明
排序为查找提供基础	有序数据可使用高效查找（如二分）
查找算法依赖数据结构	哈希表查找基于分布结构，而非顺序性
排序影响查找效率	越高效的排序，越有助于构建搜索结构（如树、索引、哈希表）

示例：

1. 数据集无序 → 只能线性查找。
2. 数据集排序 → 二分查找可在 O(log n) 完成搜索。

18.3 实例：学生成绩管理系统（Sorting + Searching）

任务描述

设计一个小程序，管理学生成绩：

- 按成绩排序；
- 查找特定学生；
- 查找前 N 名。

示例代码

```
students = [
    {"name": "Alice", "score": 85},
    {"name": "Bob", "score": 92},
    {"name": "Charlie", "score": 78},
    {"name": "David", "score": 90}
]

# 1. 排序：按成绩从高到低
students.sort(key=lambda s: s["score"], reverse=True)

# 2. 查找：按姓名搜索
def find_student(name):
    for s in students:
        if s["name"] == name:
            return s
    return None
```

```
# 3. 获取前两名
top_two = students[:2]

print("总榜：", students)
print("Bob的信息：", find_student("Bob"))
print("前两名：", top_two)
```

输出：

```
总榜： [{'name': 'Bob', 'score': 92}, {'name': 'David', 'score': 90}, ...]
Bob的信息： {'name': 'Bob', 'score': 92}
前两名： [{'name': 'Bob', 'score': 92}, {'name': 'David', 'score': 90}]
```

说明

- 排序让“最高分”“最低分”等操作简单高效；
- 查找提供按条件访问的能力；
- 两者结合构成数据管理程序的基础。

18.4 案例：商品库存系统（Product Stock System）

需求场景

电商系统中维护库存信息，需要：

- 按价格或销量排序；
- 按编码高效查找商品；
- 支持动态添加。

结构设计

使用两个视图：

1. **按价格排序的列表** → 支持二分查找（快速定位区间）
2. **哈希表字典** → 支持直接查找（按 ID）

实现示例

```
import bisect

products = [
    {"id": "A1", "price": 50},
    {"id": "B1", "price": 20},
    {"id": "C1", "price": 100},
]
```

```python
# 1. 按价格排序
products.sort(key=lambda x: x["price"])

# 2. 创建哈希索引
index = {p["id"]: p for p in products}

# 3. 二分查找价格范围
def search_by_price(low, high):
    prices = [p["price"] for p in products]
    left = bisect.bisect_left(prices, low)
    right = bisect.bisect_right(prices, high)
    return products[left:right]

# 4. 哈希表查找单个商品
def search_by_id(pid):
    return index.get(pid)

print(search_by_price(20, 60))
print(search_by_id("C1"))
```

输出：

```
[{'id': 'B1', 'price': 20}, {'id': 'A1', 'price': 50}]
{'id': 'C1', 'price': 100}
```

要点

- 二分查找在排序集合中定位区间；
- 哈希查找提供 ID 级别 O(1) 查找；
- 同时使用两者可获得搜索效率与灵活性。

18.5 案例：数据库查询优化思想

数据库中的索引（Index）本质上是查找与排序思想的结合：

- B 树 / B+ 树：维持**有序结构**实现高效“范围查找”；
- 哈希索引：用哈希表加速“等值查找”；
- 排序可优化聚合、连接等操作。

简化版数据库索引模拟

```python
records = [
    {"id": 1, "name": "Alice", "age": 25},
    {"id": 2, "name": "Bob", "age": 30},
    {"id": 3, "name": "Charlie", "age": 29},
]
```

```python
# 索引（使用字典模拟哈希表）
index = {rec["id"]: rec for rec in records}

# 查询操作
def find_by_id(record_id):
    return index.get(record_id)

print(find_by_id(2))
```

输出：

```
{'id': 2, 'name': 'Bob', 'age': 30}
```

18.6 性能比较实验（Performance Analysis）

数据规模实验

```python
import random, time

data = [random.randint(1, 10_000) for _ in range(10_000)]

# 线性查找
def linear(arr, x):
    for i, v in enumerate(arr):
        if v == x:
            return i
    return -1

# 先排序
sorted_data = sorted(data)

# 二分查找
def binary(arr, x):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == x: return mid
        if arr[mid] < x: low = mid + 1
        else: high = mid - 1
    return -1

target = random.choice(data)

t1 = time.time(); linear(data, target); t2 = time.time()
t3 = time.time(); binary(sorted_data, target); t4 = time.time()

print("线性查找耗时:", t2 - t1)
print("二分查找耗时:", t4 - t3)
```

结果示例（大致）

```
线性查找耗时: 0.0021s
二分查找耗时: 0.00004s
```

排序一次后多次查找总体效率更高。

18.7 实际开发中的技巧（Practical Tips）

目标	推荐方法
查找固定 ID	字典（哈希查找）
查找区间数据	先排序 + 二分
动态更新较多的场景	使用平衡二叉树结构（如 `bisect` + 插入）
一次排序、多次查找	预排序数据结构（如数据库索引）

第19章：数据分析与扩展库入门 (Data Analysis and Extension Libraries)

19.1 NumPy 基础（NumPy Basics）

NumPy 简介

NumPy（Numerical Python） 是 Python 数据计算的重要基础库，专注于**数组运算**、**矩阵操作**和**科学计算**。

NumPy 的核心对象：ndarray

```python
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr)
print(type(arr))
```

输出：

```
[1 2 3 4]
<class 'numpy.ndarray'>
```

数组的基本属性

```
print(arr.ndim)    # 维度数
print(arr.shape)   # 形状
print(arr.size)    # 元素数量
print(arr.dtype)   # 数据类型
```

向量化操作（Vectorized Operations）

与传统 for 循环不同，NumPy 支持**批量数学计算**。

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)
print(a * 2)
```

输出：

```
[11 22 33]
[2 4 6]
```

常用函数

目的	函数示例
创建数组	`np.arange()`, `np.linspace()`
随机数据	`np.random.rand()`, `np.random.randint()`
统计运算	`np.mean()`, `np.sum()`, `np.std()`
数组重塑	`reshape()`
连接分割	`np.concatenate()`, `np.split()`

19.2 Pandas DataFrame 操作（Pandas DataFrame Operations）

Pandas 概述

Pandas 提供了两个核心结构：

- `Series`：一维带标签数组
- `DataFrame`：二维表格（类似 Excel）

创建 DataFrame

```
import pandas as pd

data = {
    "name": ["Alice", "Bob", "Charlie"],
    "score": [85, 92, 78],
    "age": [23, 25, 22]
}
df = pd.DataFrame(data)
print(df)
```

输出:

```
       name  score  age
0     Alice     85   23
1       Bob     92   25
2  Charlie     78   22
```

基础操作

```
print(df.head())          # 查看前几行
print(df["score"])        # 选取列
print(df.loc[1, "name"])  # 按行列定位
print(df.describe())      # 统计摘要
```

条件筛选与排序

```
print(df[df["score"] > 80])
print(df.sort_values("score", ascending=False))
```

输出:

```
    name  score  age
0  Alice     85   23
1    Bob     92   25
```

新增与删除列

```
df["passed"] = df["score"] >= 80
df.drop("age", axis=1, inplace=True)
print(df)
```

输出:

```
       name  score  passed
0     Alice     85    True
1       Bob     92    True
2  Charlie     78   False
```

19.3 Matplotlib 绘图入门（Matplotlib Visualization Basics）

Matplotlib 简介

Matplotlib 是用于数据可视化的标准库，可绘制多种统计图表。

示例：绘制折线图

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

plt.plot(x, y, marker='o')
plt.title("Simple Line Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

柱形图与饼图

```
plt.bar(["A", "B", "C"], [5, 7, 3])
plt.show()

plt.pie([40, 30, 30], labels=["Cats", "Dogs", "Birds"], autopct='%1.1f%%')
plt.show()
```

美化样式

```
plt.style.use('ggplot')
plt.plot(x, y, color='purple', linestyle='--', linewidth=2)
plt.show()
```

19.4 文件与数据结合分析练习（Data Analysis with Files）

读取 CSV 文件

```
df = pd.read_csv("students.csv")
print(df.info())
print(df["score"].mean())
```

分组与聚合分析

```
grouped = df.groupby("class")["score"].mean()
print(grouped)
```

保存数据

```
df.to_excel("result.xlsx", index=False)
```

综合示例：绘制成绩分布图

```
plt.hist(df["score"], bins=10, color="skyblue", edgecolor="black")
plt.title("Score Distribution")
plt.xlabel("Score Range")
plt.ylabel("Count")
plt.show()
```

第20章：综合实践项目（Comprehensive Practice Projects）

20.1 学生成绩管理系统（Student Score Management System）

项目目标

实现一个简单的学生成绩管理应用，包含以下功能：

- 从文件读取学生数据
- 按成绩排序
- 查找学生成绩
- 导出分析结果

示例数据（students.csv）

```
name,score,subject
Alice,85,Math
Bob,91,Physics
Charlie,78,English
David,90,Math
```

代码实现

```python
import pandas as pd

# 1. 读取数据
df = pd.read_csv("students.csv")

# 2. 按成绩排序
df_sorted = df.sort_values("score", ascending=False)
print("按成绩排序：")
print(df_sorted)

# 3. 成绩查询
def search_student(name):
    result = df[df["name"] == name]
    return result if not result.empty else "未找到该学生"

# 4. 平均成绩分析
print("\n平均成绩：", df["score"].mean())

# 5. 导出结果
df_sorted.to_excel("sorted_students.xlsx", index=False)

# 测试查找函数
print("\n查询学生：", search_student("Bob"))
```

运行结果示例：

```
按成绩排序：
     name  score  subject
1     Bob     91  Physics
3   David     90     Math
0   Alice     85     Math
2 Charlie     78  English

平均成绩：  86.0
查询学生：      name  score  subject
1   Bob     91  Physics
```

20.2 文件批处理与日志分析（File Batch Processing & Log Analysis）

场景

企业日常生成的日志文件往往分布在多个目录下。本项目用于：

- 批量读取文件；
- 分析错误出现次数；
- 输出统计结果。

示例日志文件（logs/）

```
2026-04-01 INFO Application started
2026-04-01 ERROR Connection failed
2026-04-02 INFO Request received
2026-04-02 ERROR Timeout
```

代码实现

```python
import os

log_dir = "logs"
error_count = 0

# 1. 批量遍历文件
for file in os.listdir(log_dir):
    if file.endswith(".log"):
        with open(os.path.join(log_dir, file), "r", encoding="utf-8") as f:
            for line in f:
                if "ERROR" in line:
                    error_count += 1

print("错误总次数：", error_count)
```

进阶分析（错误类型统计）

```python
from collections import Counter

error_types = Counter()

for file in os.listdir(log_dir):
    if file.endswith(".log"):
        with open(os.path.join(log_dir, file), "r", encoding="utf-8") as f:
            for line in f:
                if "ERROR" in line:
```

```
                    error_type = line.split("ERROR")[-1].strip()
                    error_types[error_type] += 1

print("错误类型统计：")
for err, count in error_types.items():
    print(f"{err}: {count}")
```

输出示例：

```
错误类型统计：
Connection failed: 3
Timeout: 2
```

20.3 简单爬虫与数据提取（Basic Web Crawler & Data Extraction）

项目目标

实现一个简易爬虫：

- 抓取网页内容；
- 提取页面标题与链接；
- 输出到文件。

代码实现

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

# 获取标题
title = soup.title.text
print("网页标题：", title)

# 获取所有链接
links = [a["href"] for a in soup.find_all("a", href=True)]

# 保存到文件
with open("links.txt", "w", encoding="utf-8") as f:
    for link in links:
        f.write(link + "\n")

print("共提取链接数量：", len(links))
```

延伸任务

- 支持多网页批量爬取；
- 增加异常处理；
- 结合 Pandas 保存为 CSV。

20.4 CLI 实用工具脚本开发（CLI Utility Script Development）

任务描述

开发一个命令行工具，用于快速处理文件与数据。

目标：

- 接收命令行参数；
- 对 CSV 文件执行求和、平均等操作；
- 输出结果。

代码示例

```
import argparse
import pandas as pd

# 创建命令行解析器
parser = argparse.ArgumentParser(description="CSV 数据处理工具")
parser.add_argument("filename", help="输入 CSV 文件名")
parser.add_argument("--column", required=True, help="要分析的列名")
args = parser.parse_args()

# 读取数据
df = pd.read_csv(args.filename)
print(f"文件 {args.filename} 已加载。")

# 输出统计结果
print("平均值：", df[args.column].mean())
print("最大值：", df[args.column].max())
print("最小值：", df[args.column].min())
```

运行示例：

```
python data_tool.py students.csv --column score
```

输出：

```
文件 students.csv 已加载。
平均值：86.0
最大值：91
最小值：78
```

附录（Appendix）

本附录全面总结了 Python 学习与开发过程中可作为参考的内容，包括：
常见错误与解决方案、基础数据操作方法、排序与修改技巧、内置函数汇总、魔法方法、代码规范以及推荐学习资料。

一、常见错误类型与解决方案（Common Errors & Solutions）

错误类型	常见错误信息示例	典型原因	解决方案
SyntaxError	`SyntaxError: invalid syntax`	语法错误，如缺少冒号、括号不匹配或缩进错位	检查语句结尾是否缺少符号，如 `:` 或 `)`
IndentationError	`IndentationError: unexpected indent`	缩进空格数不一致（如混用了 Tab 和空格）	统一使用4个空格缩进
NameError	`NameError: name 'x' is not defined`	变量未定义或拼写错误	检查变量名是否定义在当前作用域
TypeError	`TypeError: unsupported operand type(s)`	对不同类型进行非法操作	使用 `int()`、`float()`、`str()` 等转换
ValueError	`ValueError: invalid literal for int()`	转换类型时数据不合法	检查输入内容，如 `"abc"` 不能转为整数
IndexError	`IndexError: list index out of range`	索引超出数组或列表范围	检查索引范围，如使用 `len()` 确认长度
KeyError	`KeyError: 'key'`	访问字典中不存在的键	使用 `dict.get()` 方法访问键值
AttributeError	`AttributeError: 'list' object has no attribute 'xxx'`	对象不具备某个属性或方法	检查类型是否正确、拼写是否准确
FileNotFoundError	`FileNotFoundError: [Errno 2] No such file`	打开的文件或路径不存在	检查路径是否正确，或用 `os.path.exists()` 验证

错误类型	常见错误信息示例	典型原因	解决方案
ZeroDivisionError	`ZeroDivisionError: division by zero`	除数为0	添加条件判断避免 0 除运算
ImportError / ModuleNotFoundError	`ImportError: No module named 'xxx'`	模块未安装或导入路径错误	使用 `pip install 模块名` 安装依赖
RuntimeError	`RuntimeError: recursion depth exceeded`	递归层数过多	使用循环替代递归或限制递归深度
UnicodeDecodeError	`UnicodeDecodeError: 'utf-8' codec can't decode`	文件编码不匹配	打开文件时使用正确编码：`open("file", encoding="utf-8")`
MemoryError	无错误堆栈但程序停止	加载超大数据或循环过多	分批处理数据、使用生成器（`yield`）优化内存
EOFError	`EOFError: EOF when reading a line`	使用 `input()` 时输入结束符	检查输入逻辑，加入默认值处理
AssertionError	`AssertionError: ...`	`assert` 条件未满足	调试条件表达式、打印中间状态确认逻辑

二、基础数据操作（Data Manipulation Basics）

1. 列表（List）

```
nums = [5, 2, 9, 1]

# 添加与删除
nums.append(6)        # 尾部添加
nums.insert(1, 3)     # 指定位置插入
nums.remove(9)        # 删除指定元素
nums.pop()            # 删除最后一个
nums.clear()          # 清空列表

# 排序与反转
nums = [5, 2, 9, 1]
nums.sort()           # 原地排序（升序）
nums.sort(reverse=True)  # 降序
nums.reverse()        # 反向排列（不排序）

# 临时排序返回新列表
sorted_list = sorted(nums)
```

---

2. 字典（Dictionary）

```
student = {"name": "Alice", "age": 23, "score": 88}

# 修改与添加
student["score"] = 92
student["major"] = "Math"

# 删除元素
del student["age"]
student.pop("score")
student.clear()

# 常用方法
print(student.keys())     # 字典所有键
print(student.values())   # 字典所有值
print(student.items())    # 键值对元组形式
print(student.get("name", "未找到"))  # 安全访问
```

---

3. 集合（Set）

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.union(b))          # 并集
print(a.intersection(b))   # 交集
print(a.difference(b))     # 差集
print(a.symmetric_difference(b))  # 对称差

a.add(6)
a.remove(1)
```

---

4. 元组（Tuple）

```
t = (10, 20, 30)

# 元组是不可变的，但可以重新整体赋值
t = t + (40,)
print(t)
```

---

5. 字符串（String）

```
text = " Python Programming "

# 基础方法
print(text.lower())        # 小写
print(text.upper())        # 大写
print(text.strip())        # 去空格
print(text.replace("Python", "Java"))
print(text.split())        # 拆分为列表
print("-".join(["A", "B", "C"]))  # 合并

# 检查方法
print(text.startswith(" "))
print(text.endswith("ing"))
```

三、数据排序与修改技巧（Sorting and Data Modification）

1. 根据条件排序

```
data = [{"name": "Alice", "score": 88},
        {"name": "Bob", "score": 95},
        {"name": "Charlie", "score": 72}]

# 按得分排序
sorted_data = sorted(data, key=lambda x: x["score"], reverse=True)
print(sorted_data)
```

---

2. 多条件排序

```
students = [
    ("Alice", 85, 22),
    ("Bob", 85, 25),
    ("Charlie", 92, 20)
]
# 按成绩降序，按年龄升序
sorted_students = sorted(students, key=lambda s: (-s[1], s[2]))
print(sorted_students)
```

---

3. 列表批量修改

```
nums = [1, 2, 3, 4]
nums = [n * 2 for n in nums]  # 列表推导式
print(nums)  # [2, 4, 6, 8]
```

---

4. Pandas 中的排序与修改

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "score": [88, 95, 72]
})

df_sorted = df.sort_values(by="score", ascending=False)
df.loc[df["score"] < 80, "remark"] = "Needs Improvement"
df["passed"] = df["score"] >= 80
print(df)
```

四、内置函数速查表（Built-in Functions Quick Reference）

分类	函数	功能
数学	`abs()`, `round()`, `pow()`, `divmod()`, `sum()`, `min()`, `max()`	各种基础数值运算
类型转换	`int()`, `float()`, `str()`, `list()`, `set()`, `tuple()`, `dict()`	数据类型互相转换
容器操作	`len()`, `sorted()`, `reversed()`, `zip()`, `enumerate()`, `filter()`, `map()`	结构操作与序列函数
输出/输入	`print()`, `input()`, `open()`	I/O 基本功能
对象操作	`type()`, `isinstance()`, `dir()`, `vars()`, `id()`, `help()`	对象与类相关工具
系统信息	`globals()`, `locals()`	查看当前作用域变量
调试控制	`assert`, `breakpoint()`	条件调试与断点

五、魔法方法一览（Magic Methods Overview）

类别	方法	功能描述
初始化与显示	`__init__`, `__repr__`, `__str__`	对象初始化与字符串表现
比较运算	`__eq__`, `__lt__`, `__gt__`, `__le__`, `__ge__`, `__ne__`	重载比较运算
算术运算	`__add__`, `__sub__`, `__mul__`, `__truediv__`	重载加减乘除等运算
容器行为	`__len__`, `__getitem__`, `__setitem__`, `__contains__`	使对象可被索引、切片
上下文管理	`__enter__`, `__exit__`	让类支持 `with` 语句
函数调用	`__call__`	让对象可被直接“调用”

类别	方法	功能描述
迭代支持	`__iter__`, `__next__`	使类可迭代
属性访问	`__getattr__`, `__setattr__`, `__delattr__`	自定义属性读写行为

六、代码规范（Python PEP 8 Style Guide）

命名参考

类型	示例	说明
模块名	`math_utils.py`	全小写，下划线分隔
类名	`DataAnalyzer`	驼峰命名法
函数名	`calculate_mean()`	小写 + 下划线
变量名	`average_score`	小写 + 下划线
常量名	`MAX_LIMIT`	全大写，下划线分隔

格式规则

- 缩进：4 个空格
- 每行最大字符数：79
- 类与函数间空两行
- 方法间空一行
- 导入顺序：标准库 → 第三方库 → 本地模块
- 字符串推荐使用双引号 `" "`

七、推荐学习资源（Recommended Learning Resources）

官方与权威文档

- Python 官方文档：https://docs.python.org/3/
- NumPy 官方文档：https://numpy.org/doc/
- Pandas 官方文档：https://pandas.pydata.org/docs/
- Matplotlib 教程：https://matplotlib.org/stable/contents.html

学习平台与练习网站

- Real Python：https://realpython.com
- LeetCode（刷题练算法）：https://leetcode.cn
- Python Tutor（可视化调试）：https://pythontutor.com
- PyPI（第三方库仓库）：https://pypi.org

推荐书籍

- 《Fluent Python（流畅的 Python）》
- 《Effective Python》
- 《Python Cookbook（第三版）》
- 《Pandas for Data Analysis》
- 《Automate the Boring Stuff with Python》